

**RÉPUBLIQUE ISLAMIQUE DE LA MAURITANIE
MINISTÈRE DE L'ENSEIGNEMENT SUPÉRIEUR
ET DE LA RECHERCHE SCIENTIFIQUE
UNIVERSITE DE NOUAKCHOTT
FACULTÉ DES SCIENCES ET TECHNIQUES
DEPARTEMENT D'INFORMATIQUE**



كلية العلوم و التقنيات
FACULTÉ DES SCIENCES ET TECHNIQUES

RAPPORT DE FIN D'ÉTUDES

En vue de l'obtention de licence en :
**Méthodes Informatiques appliquées à la gestion des Entreprises
(MIAGE)**

Sujet du projet :

**Système de monitoring et d'alerting des services et
serveurs publics**

Présenté par :
Chivae Ahmed Bezeid
C25105

Encadrant académique :
Dr.Rifaa Sadegh

Encadrant professionnel :
Eng.Babah Ivekou

Année universitaire : 2025-2026

Dédicace

Je dédie ce travail à mes parents, mon honneur et ma fierté. Ils ont été pour moi une source d'encouragement tout au long de mon parcours scolaire, en me fournissant les moyens et l'environnement nécessaires à la persévérance et à l'excellence. Je les remercie pour leurs efforts, leurs prières et leur support inconditionnel.

Je dédie également ce travail aux autres membres de ma famille, un par un, dont le soutien a nourri ma détermination et ma volonté. Ils se sont toujours montrés attentifs et prêts à m'aider à tout moment.

Enfin, je dédie ce travail à tous ceux qui y ont contribué, que ce soit directement ou indirectement.

Remerciements

Je tiens tout d'abord à exprimer mes sincères remerciements et ma gratitude à notre coordinateur, Mohamed Mahmoud Al-Bannani. Il a été pour nous un soutien et un guide tout au long de notre parcours universitaire, faisant preuve de dévouement et de rigueur dans son travail, tout en nous prodiguant conseils, orientations et encouragements ; il nous a toujours incités à élargir nos horizons de connaissance et à cultiver notre amour de l'apprentissage.

Je tiens également à exprimer ma profonde gratitude à mes superviseurs, Dr.Rifaa Sadegh et Eng.Babah Ivkou, pour l'aide, la patience et le soutien qu'ils m'ont apportés tout au long de la réalisation de ce projet. Leurs conseils ont été une source de motivation, leurs orientations un stimulant et leurs recommandations une source d'inspiration.

Je tiens aussi à remercier tous ceux qui m'ont enseigné au cours de mon parcours universitaire pour leurs efforts et leur dévouement

ملخص

يتعلق هذا المشروع بإنشاء نظام مراقبة للخدمات والسيرفرات العمومية في بلدنا. متابعة الحالة العامة للخوادم من خلال متابعة مواردهم كنسب استخدامهم والقدرة على معرفة المصدر المستخدم لهم والتنبيه عند الاقتراب من الخطر للتدخل في الوقت المناسب قبل إتلافهم أو فقدان الخدمات والبيانات الموجودة عليهم. كما يراقب هذا النظام الخدمات لمعرفة حالتهم وتسجيلها. يهدف هذا النظام إلى الحد من ظهور المشاكل وتسهيل الوصول إلى سببها في حالة وقوعها باستخدام تقنيات حديثة تسمح بالتنبيه والمراقبة في الوقت الفعلي بشكل مبسط وسهل الاستخدام.

Résumé

Ce projet consiste à mettre en place un système de surveillance des services publics et des serveurs dans notre pays.

Il suit l'état général des serveurs en surveillant la consommation des ressources — notamment les taux d'utilisation — et en identifiant les sources spécifiques de cette consommation. Il émet également des alertes à l'approche de seuils critiques, permettant une intervention rapide pour prévenir tout dommage aux serveurs ou toute perte de services et de données hébergés. Par ailleurs, le système surveille et consigne l'état des services eux-mêmes.

L'objectif du système est de réduire au minimum la survenue d'incidents et de faciliter l'identification des causes profondes en cas de problème, grâce à des technologies modernes assurant une surveillance et des alertes en temps réel via une interface épurée et conviviale.

Abstract

This project involves establishing a monitoring system for public services and servers in our country.

It tracks the general status of servers by monitoring resource usage—such as utilization rates—and identifying the specific sources consuming those resources. It also issues alerts when critical thresholds are approached, enabling timely intervention to prevent server damage or the loss of hosted services and data. Additionally, the system monitors and logs the status of the services themselves.

The system aims to minimize the occurrence of issues and facilitate the identification of root causes when problems do arise, utilizing modern technologies that provide real-time monitoring and alerts through a streamlined, user-friendly interface.

Plan

Introduction.....	1
Problématique.. ..	2
Les Objectifs mesurables	2
Présentation de l'environnement de stage.....	3
Chapitre 1 : État de l'art	5
1.1. Évolution historique	5
1.2. Les Machines Virtuelles et les Conteneurs.....	6
1.3. Les technologies de conteneurisation	9
1.3.1.Docker	9
1.3.2.Podman.....	10
1.3.3.Kubernetes	11
1.4. Les Solutions de Monitoring	12
1.4.1.Nagios.....	12
1.4.2.Zabbix.....	13
1.4.3.Prometheus.....	14
Chapitre 2 : Méthodologie	17
2.1. Critères de choix technologiques	17
2.1.1.Conteneurisation	17
2.1.2.Monitoring.....	19
2.2. Description de l'environnement expérimental ...	21
2.3. Protocol de tests et de validation	21
2.4. Métriques d'évaluation	21
Chapitre 3 : Conception et implémentation	23
3.1. Architecture globale de la solution	23
3.1.1.Serveur Central de supervision.....	23
3.1.2.Serveurs supervisés	26
3.2. Configuration	28
3.2.1.Configuration stack prometheus	28

3.2.1.1.Prometheus.yml	28
3.2.1.2.alert.rules.yml	32
3.2.1.3.alertmanager.yml	34
3.2.1.4.docker-compose.systemecentral.yml	35
3.2.2.Configuration ELK	36
3.3. Gestion de la sécurité	38
3.3.1.Création des rôles	39
3.3.2.Emettre des certificats	40
3.3.3.Extraire les certificats	41
3.3.4.Distribution des Certificats	41
3.4. Diagrammes techniques	43
3.4.1.Diagramme de séquence.....	43
3.4.2.Diagramme de déploiement.....	47
3.4.3.Diagramme de composants	48
Chapitre 4 : Expérimentation et résultats	50
4.1. Tableau de bord Grafana	51
4.2. Tableau de bord Kibana	52
4.3. Tests de charge et analyse de performance	52
Chapitre 5 : Discussion	63
4.1. Limites de l'approche proposée	64
4.2. Problèmes rencontrés et solutions apportées.....	65
4.3. Scalabilité et haute disponibilité	66
Conclusion et perspectives.....	68
Références.....	70
Webographie	70

Introduction Générale

Introduction

Aujourd'hui, nous constatons les progrès réalisés par la Mauritanie dans le domaine numérique et la généralisation de son utilisation dans la plupart des secteurs et des institutions, ce qui suscite un engouement de la part des citoyens et une utilisation croissante des services publics numériques. C'est là que surgissent les problèmes : augmentation de la charge, multiplication des demandes, délais de réponse trop longs pour les usagers et panne des services ! Cela soulève la question des moyens permettant de détecter les risques avant que des problèmes surviennent. Par exemple, Lorsque le nombre de requêtes adressées à un serveur hébergeant certains services augmente, le taux d'utilisation des ressources s'accroît, et en cas d'erreur, le temps nécessaire pour en déterminer la cause en consultant les logs de plusieurs services s'allonge. Ce qui peut entraîner un arrêt complet et une panne ! Plutôt que d'être pris au dépourvu, nous tenterons dans ce projet de mettre en place un système nous permettant de consulter quotidiennement les logs des services et de surveiller l'état général des ressources vitales des serveurs.

Problématique

La transition vers le numérique des services publics entraînera inévitablement une augmentation du nombre de demandes. Et comme il s'agit de services publics, il est essentiel de garantir qu'ils ne soient interrompus ou perturbés, ne serait-ce qu'une minute. C'est pourquoi, Il n'est pas raisonnable d'attendre qu'une erreur se produise pour réexaminer nos ressources, pas plus qu'il n'est possible de passer au crible des milliers de logs pour déterminer la cause d'un problème qui se produire. Dans ce cas, les citoyens seraient alors confrontés à des retards dans le traitement de leurs demandes et se heurteraient à des erreurs, ce qui pourrait nuire à l'opinion des clients quant à la capacité des services numériques à leur faciliter la vie.

De plus, l'absence de contrôle et de suivi pourrait entraîner la destruction totale des serveurs et des services, ce qui pourrait à son tour entraîner la perte des archives de l'État, des documents officiels et des données des citoyens.

Les Objectifs mesurables

La surveillance des serveurs publics nous permet de rester informés en permanence de l'état général des ressources des

serveurs en temps réel, ce qui nous permet d'intervenir à temps avant que des problèmes ne surviennent. Cela nous permet ainsi de limiter les erreurs chez l'utilisateur, les risques de perte de données et de détérioration des serveurs.

De plus, la surveillance des services publics permet d'analyser les performances de ces services, de détecter plus rapidement les erreurs, et d'évaluer l'expérience des utilisateurs.

Présentation de l'environnement de stage

Dans le cadre de notre projet de fin d'études, nous avons effectué un stage au sein du **Ministère de la Transition Numérique, de l'Innovation et de la Modernisation de l'Administration (MTNIMA)**, dont la mission générale consiste à élaborer, organiser et mesurer les politiques nationales dans les trois domaines suivants : « la transformation numérique », « l'innovation » et « la modernisation de l'administration ».

La structure de ce ministère comprend le cabinet du ministre, le secrétariat général et les directions centrales, qui regroupent huit directions :

- La direction des affaires Juridiques (DJ)
- La direction de la stratégie et de la Coopération (DSC)
- La direction des infrastructure (DI)
- La Direction de l'administration des Systèmes et de la Sécurité (DAS)

- La Direction du Développement et de l'interopérabilité (DDI)
- Direction de l'Innovation
- Direction de la Modernisation de l'Administration (DMA)
- Direction des Ressources (DR)

La Direction de l'Administration des systèmes et de la Sécurité, chargée de ce projet, se compose de deux services : le service de gestion des systèmes et le service de la sécurité.

La mission du service de l'Administration du système consiste à gérer, contrôler et superviser les services informatiques de l'administration, en veillant à la mise en œuvre des opérations de maintenance du matériel et des logiciels, en traitant les incidents de premier niveau et en fournissant une assistance technique aux utilisateurs.

Chapitre 1 : État de l'art

Introduction

Dans ce chapitre, nous aborderons les évolutions dans le domaine des machines virtuelles et des conteneurs, depuis leurs débuts jusqu'à la situation actuelle, puis nous présenterons un aperçu général des technologies liées aux conteneurs et à la surveillance, en passant en revue certaines des technologies utilisées dans ces deux domaines.

1.1. Évolution historique

Au milieu des années 1960, le centre technologique du géant de l'informatique IBM à Cambridge en collaboration avec le MIT, à développer CP40 et CP-67, débuts du concept de “virtualisation” qui est définie comme une technologie qui consiste à créer des environnements virtuels à partir d'une seule machine physique. Et ensuite a introduit VM/370 dans les années 1970, Exécution de la virtualisation sur le système /370 [1], tandis que Unix a lancé l'appel système Chroot en 1979, qui a permis d'isoler les processus et a influencé la technologie des conteneurs plusieurs décennies plus tard [2]. Qu'est une méthodologie consistant à encapsuler une application et ses éléments associés (le code logiciel et les ressources nécessaires) dans une unité indépendante appelée “Conteneur”.

Les années 1990 ont vu l'avènement de la virtualisation sur PC comme Connectix Virtual PC, JVM, et VMware [3].

Les années 2000 ont vu l'apparition de la virtualisation au niveau du système d'exploitation, le noyau permettant d'avoir plusieurs instances isolées de l'espace utilisateur, grâce à FreeBSD Jails, OpenVZ et Linux Containers (LXC), ce qui a ouvert la voie à la virtualisation légère [4].

En 2010, Docker a révolutionné le processus de déploiement des applications en rendant les conteneurs portables et faciles à utiliser pour les développeurs.

Lorsque Docker a lancé Docker Engine en 2013, celui-ci a normalisé l'utilisation des conteneurs avec des outils faciles à utiliser pour les développeurs [5].

Aujourd'hui, il existe divers outils et plateformes de conteneurisation qui prennent en charge les normes de l'Open Container Initiative établies par Docker, notamment Podman, Buildah et Skopeo.

Mais c'est alors qu'un problème s'est posé : Comment automatiser la gestion d'un grand nombre de conteneurs ?

Kubernetes introduit en 2014, Après que trois ingénieurs de Google ont proposé l'idée de créer un système open source de gestion des conteneurs, est devenu la norme pour l'orchestration des conteneurs [6].

1.2. Les Machines Virtuelles et les Conteneurs

Les machines virtuelles et les conteneurs sont des technologies qui permettent de rendre les applications autonomes par rapport aux ressources de l'infrastructure informatique.

Une machine virtuelle est un réplica numérique d'une machine physique. Nous pouvons disposer de plusieurs machines virtuelles, chacune dotée de son propre système d'exploitation, fonctionnant sur le même système d'exploitation hôte. De plus, nous pouvons créer une machine virtuelle contenant tous les éléments requis pour faire exécuter notre logiciel.

Un conteneur est un ensemble de codes de programmation comprenant le code de l'application, ses bibliothèques et ses autres dépendances. La technologie des conteneurs permet de transférer facilement les applications, de sorte que le même code puisse fonctionner sur n'importe quel appareil.

La technologie de virtualisation a été développée afin d'exploiter efficacement la capacité matérielle croissante et la puissance de calcul en constante augmentation. La virtualisation permet d'installer plusieurs systèmes d'exploitation et de créer plusieurs environnements sur un même matériel. Les conteneurs, quant à eux, sont conçus pour emballer et exécuter des applications de façon reproductible dans divers environnements. Au plutôt que de recréer l'environnement, l'application est emballée pour fonctionner sur tous les types d'environnements, qu'ils soient physiques ou virtuels.

Les deux permettent une isolation totale des applications, ce qui vous permet de les exécuter dans divers environnements. Elles

isolent également l'infrastructure sous-jacente, de manière à ce que les utilisateurs n'aient pas à s'en soucier. Cependant, le rôle des conteneurs et des machines virtuelles, ainsi que leur taux d'utilisation, varient en fonction du lieu et du mode de déploiement de l'application. Les conteneurs permettent de virtualiser le système d'exploitation, ce qui permet à l'application de fonctionner de manière autonome sur n'importe quelle plateforme. Les machines virtuelles vont plus loin en virtualisant le matériel, ce qui vous aide à exploiter pleinement vos ressources matérielles.

Caractéristiques	Virtualisation	Containerisation
Mode de fonctionnement	Consiste à installer un logiciel de virtualisation sur une machine physique. Nous pouvons configurer et mettre à jour le système d'exploitation invité et ses applications sans effectuer le système d'exploitation hôte.	Consiste à créer des progiciels autosuffisants qui fonctionnent de manière cohérente, indépendamment des machines sur lesquelles ils sont exécutés.
Taille	Les fichiers image sont plus volumineux, car ils contiennent leur propre système d'exploitation.	Les fichiers image sont plus légers, car les conteneurs ne regroupent que les ressources nécessaires à l'exécution d'une seule application.
Configuration de l'environnement	Installer manuellement des logiciels système, prendre des clichés des états de configuration et les restaurer à un état antérieur si nécessaire.	Fournissent des définitions statiques des configurations.
Flexibilité	Moins flexible. La migration	Plus flexible. Nous

	comporte des défis.	pouvons rapidement migrer entre des environnements.
--	---------------------	---

1.3. Les technologies de conteneurisation

Il existe de nombreuses technologies spécialisées dans la création, la gestion et le déploiement de conteneurs, qui se distinguent par leur mode de configuration et d'utilisation. Parmi lesquelles :

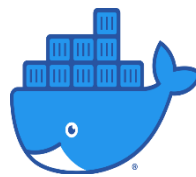
1.3.1. Docker

Docker est une plateforme logicielle standard et open source qui aide les développeurs à créer, tester et déployer rapidement des applications conteneurisées. Vous pouvez exécuter de nombreux conteneurs Docker, chacun avec sa propre application, sur un seul serveur, et ses applications seront isolées les unes des autres, ce qui garantit la sécurité de données et la fiabilité.

Docker a été présenté pour la première fois par l'ingénieur Solomon Hykes, qui a exposé ce concept lors de la conférence PyCon 2013.

Aujourd'hui, les conteneurs Docker sont utilisés pour des déploiements critiques à grande échelle. Inspiré par le concept

fondamental de la conteneurisation, Docker a apporté une approche nouvelle et innovante du déploiement applicatif. Il a fait passer la conteneurisation à un niveau supérieur en introduisant un ensemble de fonctionnalités puissantes. La plupart du temps, les conteneurs Docker peuvent être déployés sur n'importe quel serveur, ou ordinateur portable ou de bureau, exécutant ce système d'exploitation, sans nécessiter de modification de configuration.



Logo docker

1.3.2. Podman

Podman est un outil Open Source qui sert à développer, gérer et exécuter des conteneurs. Développé par des ingénieurs Red Hat à partir de 2017 en collaboration avec la communauté open source.

Podman ressemble les conteneurs dans des pod; Un pod est un groupe de conteneurs qui s'exécutent ensemble et partagent les mêmes ressources.

Podman se démarque des autres moteurs de conteneurs, car il n'a pas de démon, ce qui permet notamment l'exécution des conteneurs sans nécessiter les privilèges root.

Podman a révolutionné le monde des conteneurs en offrant les mêmes capacités performantes que les moteurs de conteneurisation leaders sur le marché, avec des fonctionnalités de flexibilité, d'accessibilité et de sécurité qui répondent aux attentes de nombreuses équipes de développement.



1.3.3. Kubernetes

Kubernetes est une plateforme Open source qui permet l'automatisation du déploiement et la gestion des conteneurs, développée par des ingénieurs de Google en 2014.

Offre la mise à l'échelle automatique, la gestion des versions et des mises à jour, ainsi que l'équilibrage de charge s'il nécessaire.

Le Pod, qui représente l'unité de déploiement de conteneur sur Kubernetes, permet le déploiement de plusieurs conteneurs interconnectés.



Caractéristiques	Docker	Podman	Kubernetes
Architecture	Daemon	Sans daemon	Pas de daemon central
Unités de déploiement	Conteneur	Conteneur	Pod
Portée	Serveur unique	Serveur unique	Système distribué

1.4. Les Solutions de Monitoring

Le monitoring de serveurs joue un rôle important. Permet d'observer en permanence l'environnement du serveur, Ce processus permet la détection des problèmes ce qui signifie que vous pouvez traiter les problèmes potentiels avant qu'ils ne se transforment en crises majeures affectant les fonctions normales des serveurs. Ils existent plusieurs outils de monitoring parmi lesquelles

1.4.1. Nagios

Nagios : est un logiciel ordonnanceur qui surveille les systèmes, les réseaux et l'infrastructure. Nagios offre des services de surveillance et d'alerte pour les serveurs, les applications et les services. Il alerte les utilisateurs en cas d'incidents et les avertit une deuxième fois lorsque le problème a été résolu.

Nagios a été le premier outil de monitoring IT open source à se positionner en 1999.

Nagios permet d'identifier les défaillances dans un système informatique, comme un serveur en surcharge ou un service inactif. Il est utilisé dans les entreprises pour assurer le bon fonctionnement des systèmes critiques.

The Nagios logo consists of the word "Nagios" in a bold, black, sans-serif font. The letter "N" is stylized with a horizontal bar extending to the left.

Logo nagios

1.4.2. Zabbix

Zabbix est un logiciel open-source développé par Zabbix LLC en 2001. Permet de superviser de nombreux paramètres réseaux ainsi que la santé et l'intégrité des serveurs et ainsi utilise un mécanisme de notification flexible qui permet aux utilisateurs de configurer une base d'alerte e-mail pour pratiquement tous les événements. Cela permet une réponse rapide aux problèmes des serveurs.

The ZABBIX logo features the word "ZABBIX" in white, uppercase, sans-serif font, centered within a solid red rectangular background.

Logo Zabbix

1.4.3. Prometheus

Prometheus est un ensemble d'outils open source de surveillance informatique et générateur d'alertes maintenu par la Cloud Native Computing Foundation. Il enregistre des métriques en temps réel dans une base de données temporelles.

Prometheus fournit un langage de requête appelé PromQL pour interroger vos données de séries temporelles. De nombreux projets utilisent PromQL, y compris Grafana et Alertmanager, pour aider dans des tâches analytiques et opérationnelles telles que la visualisation des données et la création d'alertes.



Logo Prometheus

1.4.4. ELK

L'acronyme ELK désigne trois logiciels : Elasticsearch, Logstash et Kibana, qui sont des outils permettant de collecter, d'analyser et de visualiser des logs, développés par la société Elastic.

Chacun de ces outils à sa propre fonction : Logstash collecte les logs, les transforme et les envoie à Elasticsearch, qui se charge de les stocker et de les indexer, et Kibana offre une

représentation visuelle de ces logs stockés dans Elasticsearch.

Grâce à cet ensemble d'outils, nous pouvons facilement surveiller nos services en suivant les logs en temps réel, avec la possibilité d'effectuer des recherches et d'appliquer des filtres.



Logo Elastic

Caractéristique	Nagios	Zabbix	Prometheus	Elk
Objectif	Vérifier l'état du service	Surveillance complète	Collecte et surveillance des métriques	Collecte et analyse des logs
Type de données traitées	Statuts checks	Métriques	Métriques	Logs
Méthode de collecte de données	Plugin	Agent	Exporter	Agent

Conclusion

Ce chapitre a donné un panorama général des concepts de virtualisation et de Conteneurisation, ainsi que de la surveillance, en mettant en lumière leurs principales techniques à travers leur définition et leur comparaison.

Chapitre 2 : Méthodologie

Introduction

Ce chapitre portera sur le choix des outils adaptés à la réalisation de ce projet, en fonction de critères précis permettant de les sélectionner parmi les différentes alternatives, ainsi que sur la définition de l'environnement dans lequel il sera mis en œuvre, sans oublier les méthodes de test et de validation.

2.1. Critères de choix technologiques

2.1.1. Conteneurisation

Les outils de conteneurisation étant aujourd'hui nombreux et chacun d'entre eux présentant ses propres caractéristiques et avantages, le choix entre eux doit reposer sur des critères précis afin de sélectionner celui qui convient le mieux à notre projet.

Critère	Docker	Podman	Kubernetes
La simplicité de déploiement des outils	Docker permet de déployer rapidement les outils à l'aide d'images prêtes à l'emploi et de fichiers Docker Compose.	Podman offre un mécanisme de déploiement similaire à Docker, mais certaines documentations et configurations disponibles sont moins nombreuses, ce	Le déploiement nécessite la mise en place d'un cluster ainsi que la configuration de plusieurs composants (Pods, Services, Deployments, Ingress, etc.).

		qui peut augmenter légèrement le temps de mise en œuvre.	
Documentation	Docker dispose d'une documentation riche, d'une vaste communauté et d'un grand nombre de tutoriels, d'images officielles.	Podman possède une documentation de qualité, mais son écosystème reste moins développé.	Kubernetes dispose d'une documentation très complète et d'une large communauté.
Temps de mise en œuvre	Docker permet une mise en œuvre rapide grâce à des commandes simples et standardisées ainsi qu'à Docker Compose, qui permet de lancer plusieurs services simultanément à partir d'un seul fichier de configuration.	Podman permet une mise en œuvre rapide grâce à des fonctionnalités similaires à Docker Compose via les pods ou des outils comme podman-compose, permettant le déploiement de plusieurs services à partir d'une configuration unique.	La création et la configuration d'un cluster nécessitent davantage de temps et d'efforts.

D'après cette étude, nous choisissons docker comme un outil de conteneurisation. Grâce auquel nous allons exécuter les outils de notre système dans de conteneurs.

2.1.2. Monitoring

Parmi les différents outils de surveillance, le choix repose sur les exigences du projet, selon des critères permettant de sélectionner les outils les plus adaptés.

Critère	Prometheus & Grafana	Zabbix	Nagios
Le coût	Open source	Open source	Open source
Scalabilité	Prometheus propose des solutions favorisant l'évolutivité, telles que la Fédération, Service Discovery...	Zabbix propose mécanisme pour favoriser la scalabilité comme Zabbix proxy, auto-registraton, Discovery...	Dans Nagios, la scalabilité est limitée, il ne convient qu'aux petits environnements car chaque check exécute de nombreux Plugins, ce qui entraîne une forte consommation des ressources lorsque le nombre de hots surveillés augmente
Alerting	Prometheus fournit avec Alertmanager un système d'alerte efficace qui permet une gestion et une organisation flexibles des alertes avec une routage des notifications par e-mail, télégramme, Slack...	Zabbix offre un système d'alerte interne flexible qui permet de créer et de configurer des alertes, ainsi que d'envoyer des notifications par e-mail, SMS et autres moyens	Nagios dispose d'un système d'alerte performant qui permet de créer des alertes et de les envoyer par sms, e-mail, Télégramme et d'autres moyens. Il offre également la possibilité d'envoyer les alertes vers plusieurs canaux,

			ce qui garantit une réponse rapide
Visualisation	L'intégration de Prometheus à Grafana offre une excellente visualisation des métriques, car Grafana propose des tableaux de bord modernes, conviviales et faciles à comprendre	Zabbix propose de bons tableaux de bords intégrées, mais les options de personnalisation et d'interaction sont limitées	Nagios privilégie la surveillance plutôt que la présentation visuelle, avec des tableaux de bord obsolètes et d'une qualité relativement médiocre.
Documentation	Prometheus propose une documentation complète qui permet notamment de comprendre tous les aspects de cette technologie, depuis ses paramètres jusqu'à ses utilisations et sa personnalisation.	Zabbix fournit une documentation complète contenant les références et les détails nécessaires à l'utilisateur	La documentation de Nagios couvre les concepts de base et fournit des exemples, mais elle n'est pas à jour et la navigation entre les documents n'est pas fluide

Sur la base de cette analyse, nous avons choisi Prometheus et Grafana comme outils de surveillance des serveurs, et ELK pour la surveillance des services. Ces outils nous permettent d'assurer une surveillance efficace.

2.2. Description de l'environnement expérimental

L'environnement expérimental de ce projet se compose de conteneurs Docker s'exécutant sur un hôte Ubuntu, ainsi que de serveurs fonctionnant sous divers systèmes d'exploitation et dotés de ressources variées, à partir desquelles des métriques et des journaux sont collectés.

2.3. Protocol de tests et de validation

Pour vérifier et valider ce système, nous utiliserons un serveur sur lequel nous déploierons une application web avec une base de données, puis nous les surveillerons en temps réel.

Ensuite nous effectuerons des tests de charge pour simuler une charge et observer le comportement des métriques et des logs.

2.4. Métriques d'évaluation

- Le système fonctionne sans problème et sans erreur de configuration.
- Toutes les métriques et les logs sont collectées et visualisées clairement en temps réel.
- Les alertes doivent parvenir aux administrateurs en temps opportun.
- Détection simple et rapide des erreurs et des risques.

- Le système doit être évolutif et capable de prendre en charge un nombre croissant de serveurs et services surveillés.

Conclusion

Dans cette section, nous avons sélectionné les meilleures technologies pour la mise en œuvre de ce projet, défini des méthodes pour le tester et établi des critères permettant de vérifier sa validité.

Chapitre 3 : Conception et implémentation

Introduction

Dans ce chapitre, nous aborderons l'architecture générale du projet, la création et la description de ses fichiers de configuration, puis nous réaliserons des schémas illustrant son fonctionnement et ses composants.

3.1. Architecture globale de la solution

Dans le cadre de ce projet, nous allons mettre en place un système de surveillance centralisé qui permettra de collecter l'ensemble des logs de service et des métriques de performance des serveurs, puis de les regrouper et de les afficher sur un seul serveur central. Cela nous permettra de surveiller en temps réel les performances des différents serveurs.

3.1.1. Serveur Central de supervision

Ce serveur constitue la base du système, nous y exécuterons les outils de surveillance suivants :

○ Surveillance des serveurs

Prometheus : grâce à son architecture basée sur un modèle de type “pull”, Prometheus est responsable de la récupération, le stockage et l’analyse des métriques auprès des serveurs qu’il surveille :

Retrieval : Prometheus utilise un modèle de type “pull” dans lequel le serveur Prometheus interroge les cibles qu’il surveille pour obtenir des métriques au lieu d’attendre qu’elles soient transmises.

Prometheus intègre un Service Discovery permet de l’automatisation de la détection des cibles vers les objectifs à surveiller ; À chacun de ces cibles, Prometheus envoie une requête HTTP GET. Une fois les métriques obtenues, il les convertit en série temporelle.

TSDB : Les données collectées sont stockées avec leurs labels correspondants dans une base de données locale conçue pour les données de séries temporelles, ce qui permet une recherche et une agrégation rapides des données.

PromQL : la base de données stockées est optimisée pour les opérations de requête rapides et efficaces avec le langage PromQL (Prometheus Query Language). Avec promQL nous pouvons écrire des expressions et des filtres pour extraire des informations spécifiques, effectuer des agrégations, des

regroupements, des jointures et d'autres opérations sur les métriques collectées

Grafana : Pour transformer les données collectées par prometheus en visualisation significatives et compréhensibles, Grafana permet de créer des tableaux de bord pour afficher et analyser des métriques. Les tableaux de bord peuvent inclure des graphiques, des jauges, des tableaux, des cartes et d'autres types de visualisation.

Alertmanager : PromQl nous permet également de définir des alertes(déclencheurs). Nous pouvons spécifier une expression PromQl et définir un seuil. Lorsque ce seuil est atteint, le déclencheur est activé et une notification est générée. Prometheus n'envoie pas de notifications vers des messageries instantanées ou par e-mail, il génère simplement une requête push vers un système externe. Prometheus intègre une application distance, appelée Alertmanager, qui remplit cette fonction.

Alertmanager reçoit les notifications de Prometheus et gère toute la logique nécessaire pour dupliquer les alertes, désactiver celles qui ne sont pas nécessaires. Les alertes peuvent ensuite être envoyées pour notifier via de messagerie (Slack, Télégramme...) ou par e-mail.

○ Surveillance des services

Logstash : reçoit les logs au format texte, les analyse, en extrait les champs identifiables puis les filtre, et les envoie ensuite à Elasticsearch au format JSON.

Elasticsearch : récupère les données provenant de Logstash, les stocke et les indexe, ce qui facilite leur recherche.

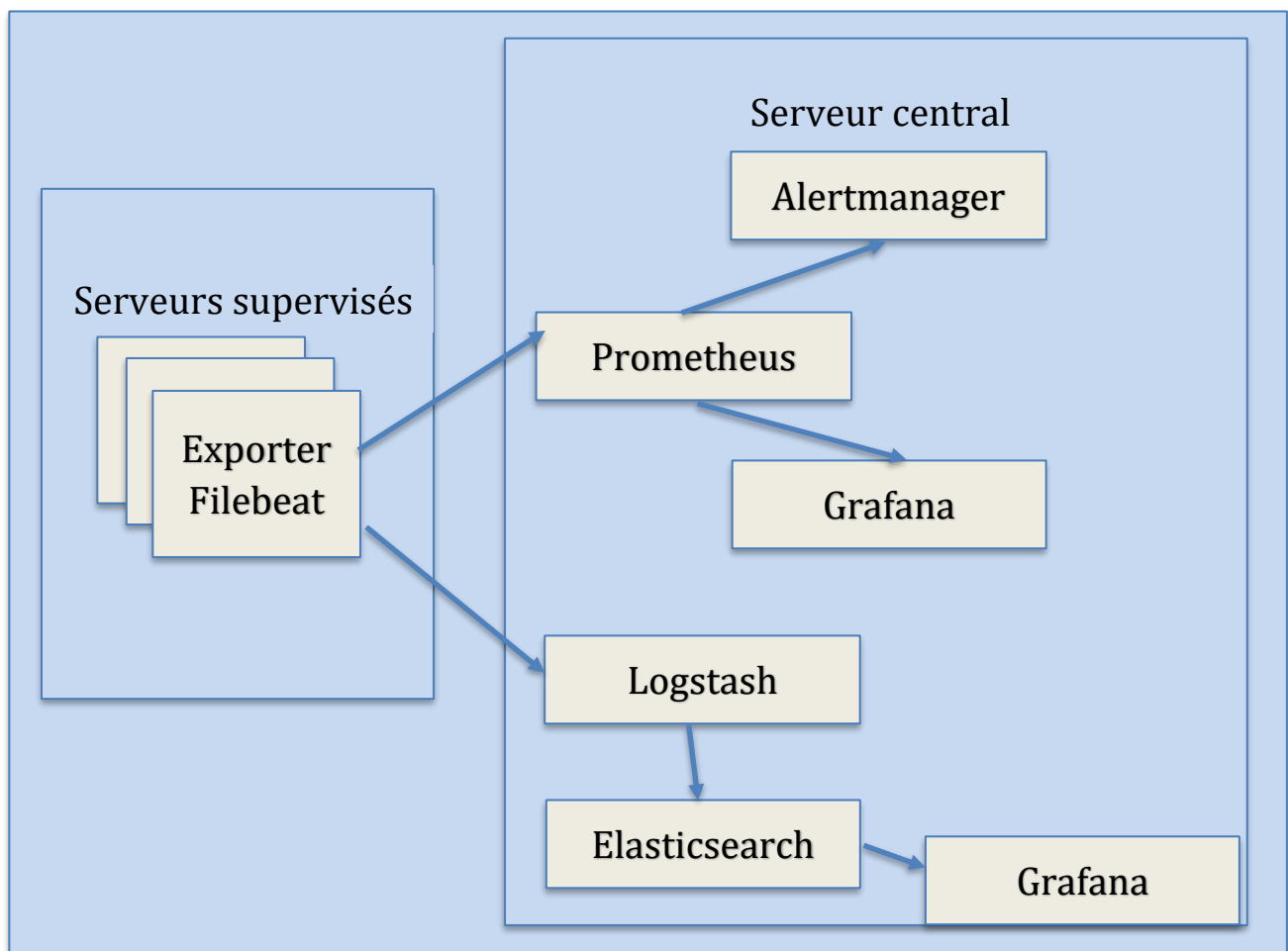
Kibana : affiche les données enregistrées dans Elasticsearch sous forme des tableaux de bords faciles à comprendre et à surveiller, et permet également d'effectuer des recherches à l'aide du langage KQL, spécialement conçu pour la recherche dans les données Elasticsearch.

3.1.2. Serveurs supervisés

Pour que Prometheus puisse collecter les métriques, nous devons installer un exporter sur chacun de ces serveurs selon son système d'exploitation Node Exporter pour linux et MacOS, Windows Exporter pour Windows.... Ce logiciel sert de passerelle entre Prometheus et notre serveur sous-jacent. L'architecture de ces exporters se compose de collecteurs qui peuvent être activés

ou désactivés de manière sélective ; Lorsque l'exporter reçoit une requête HTTP GET à l'adresse indiquée dans Prometheus, Les collecteurs listent et rassemblent les données du système, puis les convertissent en métriques et les envoient via une requête HTTP POST vers Prometheus.

Pour collecter les logs, nous installons **Filebeat** sur le serveur hébergeant les services surveillés. Filebeat recherche les fichiers de log dans le système, les surveille en continu et envoie leur contenu à Logstash. Ainsi, lorsqu'une nouvelle ligne est ajoutée, il l'envoie uniquement sans retransmettre les données déjà envoyées.



3.2. Configuration

3.2.1. Configuration stack prometheus

3.2.1.1.Prometheus.yml

Pour configurer Prometheus, nous commençons par créer le fichier `prometheus.yml`, qui constitue le cœur de la configuration. Ce fichier contiendra les paramètres liés au fonctionnement de Prometheus : comment il trouve le chemin vers les cibles, comment il collecte les métriques, ainsi d'autres paramètres.

La structure de ce fichier est divisée en plusieurs sections, chaque section contient des paramètres avec ces valeurs.

`prometheus.yml`

```
global:
  scrape_interval: 5s

rule_files:
  - /etc/prometheus/alert.rules.yml

alerting:
  alertmanagers:
    - static_configs:
      - targets: ['alertmanager:9093']
```

```
scrape_configs
- job_name: "node-exporter"
  file_sd_configs:
    - files:
      - /etc/prometheus/targets.json
    refresh_interval: 5s
```

Section	Paramètres	Justification de choisi de valeur
Section 1: global Définit les paramètres qui s'appliquent à tous les objectifs.	scrape_interval: délai entre deux scrapes; en d'autres termes, une fois ce délai écoulé, Prometheus enverra régulièrement des requêtes HTTP GET à tous les jobs	La demande en serveurs publics étant très forte, les données et les indicateurs évoluent rapidement. C'est pourquoi il convient d'aborder les processus de scraping en veillant à ne pas surconsommer l'espace de stockage et les ressources
	scrape_timeout: Délai d'attente de Prometheus pour obtenir une réponse lors de l'envoi d'une	La demande en serveurs public étant très forte, les données et les indicateurs évoluent rapidement.

	requête HTTP Get. Si ce délai expire avant l'obtention de la réponse l'opération est annulée	C'est pourquoi il convient d'aborder les processus de scraping en veillant à ne pas surconsommer l'espace de stockage et les ressources
	évaluation_interval: C'est l'intervalle de temps après lequel Prometheus vérifie régulièrement les règles d'alerte.	Le mieux est de la régler à la même valeur que scrape_interval afin de vérifier les données dès leur réception
	extra_scrape_metrics: Ce paramètre permet d'enregistrer des métriques relatives au processus de scraping lui-même, telles que la durée qu'il a prise, la consommation de ressources...	La valeur de ce paramètre est de type booléen ; il faut donc lui attribuer la valeur "true" pour l'activer.
Section 2 : rule_files	Seulement le path vers le fichier qui	Le vrai path vers les conditions

Cette section indique le chemin d'accès au fichier contenant les règles d'alerte.	contient les conditions d'alertes	
Section 3 : Alerting Cette section concerne la configuration de l'envoi des alertes détectées dans Prometheus vers Alertmanager en définissant le chemin d'accès à celui-ci, en indiquant le nom du service et le port sur lequel il fonctionne.	Targets : défini le path vers alertmanager	Il suffit d'ajouter le nom du service et le port du Alertmanager
Section 4 : scrape_config Nous indiquons ici les adresses des jobs sur lesquels Prometheus effectue le scraping.	file_sd_configs: Elle permet d'activer le mécanisme Service Discovery, qui nous offre la possibilité de faire en sorte que Prometheus lise les targets à partir de fichiers externes.	Nous créons le fichier targets.json qui contiendra la liste des targets Chaque targets correspond à l'adresse ip du serveur surveillé ainsi qu'au port sur lequel le Node Exporter y fonctionne.

	job_name : nous permet de regrouper plusieurs serveurs sous un même nom.	Il suffit de donner un nom clair.
--	--	-----------------------------------

3.2.1.2.alert.rules.yml

Il s'agit du fichier dans lequel nous définissons les règles d'alerte ; Prometheus le consulte pour vérifier les conditions et déclenche une alerte lorsqu'elles sont remplies.

alert.rules.yml

```
groups:
  - name: SystemAlerts
    rules:
      - alert: NodeDown
        expr: up == 0
        for: 15s
        labels:
          severity: critical
        annotations:
          summary: "Node Down"
          description: "Le node demandé est inaccessible"

      - alert: HighCPUUsage
        expr: 100 - (avg by(instance)
(rate(node_cpu_seconds_total{mode="idle"}[1m])) * 100) > 80
```

```

for: 2m
labels:
  severity: critical
annotations:
  summary: "High CPU Usage"
  description: "L'utilisation du CPU a augmenté de 80%"

- alert: HighMemoryUsage
  expr: (1 - (node_memory_MemAvailable_bytes /
node_memory_MemTotal_bytes)) * 100 > 80
  for: 2m
  labels:
    severity: warning
  annotations:
    summary: "High Memory Usage"

```

Paramètre	Description
Alert	Identifier l'alerte par son nom. Nous donnons un nom qui précise à quoi se rapporte l'alerte.
Expr	Définition d'une condition à l'aide de PromQl
For	Le délai entre la détection de la condition et l'envoi de l'alerte. Deux minutes d'attente pour s'assurer que la condition est remplie
Labels	Ajouter une description de l'alerte elle-même
Annotation	Ajouter des informations explicatives pour clarifier l'alerte lors de l'envoi d'une notification

Le CPU, le disque dur et la RAM constituent les principales ressources des serveurs, ce qui en fait la cause principale de leurs

problèmes. C'est pourquoi nous avons mis en place des alertes qui se déclenchent lorsqu'ils s'approchent de la zone de risque.

3.2.1.3.alertmanager.yml

C'est là que nous définissons où Alertmanager enverra les notifications.

alertmanager.yml

```
global:
  smtp_smarthost: 'smtp.gmail.com:587'
  smtp_from: 'notre-email@gmail.com'
  smtp_auth_username: 'notre-email@gmail.com'
  smtp_auth_password: 'xxxx xxxx xxxx xxxx'

route:
  receiver: 'email-team'

receivers:
  - name: 'email-team'
    email_configs:
      - to: 'notre-email@gmail.com'
        send_resolved: true
```

Étant donné que “email” est le moyen de communication le plus utilisé dans les organismes publics, nous l'avons désignée comme destinataire des notifications d'AlertManager.

3.2.1.4.docker-compose.systemecentral.yml

docker-compose.yml

```
services:

  prometheus_central:
    image: prom/prometheus
    container_name: prometheus_central
    ports:
      - "127.0.0.1:9090:9090"
    volumes:
      -
        ./PrometheusCentral/prometheus.yml:/etc/prometheus/prometheus.
        yml
      -
        ./PrometheusCentral/targets.json:/etc/prometheus/targets.json
      -
        ./PrometheusCentral/alert.rules.yml:/etc/prometheus/alert.rule
        s.yml
      -
        ./PrometheusCentral/servicetargets.json:/etc/prometheus/servi
        cestargets.json
    command:
      - '--config.file=/etc/prometheus/prometheus.yml'
    restart: unless-stopped

  grafana:
    image: grafana/grafana
    container_name: grafana
    ports:
      - "127.0.0.1:3000:3000"
    restart: unless-stopped

  alertmanager:
    image: prom/alertmanager
    container_name: alertmanager
    ports:
      - "127.0.0.1:9093:9093"
    volumes:
      -
```

```
./Alertmanager/alertmanager.yml:/etc/alertmanager/alertmanager
.yml
  command:
    - '--config.file=/etc/alertmanager/alertmanager.yml'
  restart: unless-stopped
```

Paramètre	Description
Image	Le Template utilisé pour démarrer le conteneur
container_name	Identifier le conteneur
Volumes	Fournir des fichiers de configuration au conteneur
Port	Relier le port du serveur au port du conteneur, fournir une interface pour le service.
Restart	Pour démarrer le service au démarrage du serveur

Pour chaque service, nous avons utilisé l'image officielle de Prometheus et le port par défaut.

3.2.2. Configuration ELK

3.2.2.1. docker-compose.elk.yml

Dans ce fichier, nous définissons les services ELK et les connexions entre eux, tout en ajoutant des configurations pour chacun d'eux.

Docker-compose.elk.yml

```
services:
```

```
elasticsearch:
  image: docker.elastic.co/elasticsearch/elasticsearch:7.14.0
  container_name: elasticsearch
  environment:
    - discovery.type=single-node
    - xpack.security.enabled=false
  volumes:
    - esdata:/usr/share/elasticsearch/data
  ports:
    - "9200:9200"
```

```
logstash:
  image: docker.elastic.co/logstash/logstash:7.14.0
  container_name: logstash
  volumes:
    - ./logstash/pipeline:/usr/share/logstash/pipeline
  ports:
    - "5044:5044"
  depends_on:
    - elasticsearch
```

```
kibana:
  image: docker.elastic.co/kibana/kibana:7.14.0
  container_name: kibana
  environment:
    - ELASTICSEARCH_URL=http://elasticsearch:9200
  ports:
    - "5601:5601"
  depends_on:
    - elasticsearch
```

```
volumes:
  esdata:
    driver: local
```


3.2.2.2.logstash.conf

Dans ce fichier, nous définissons le port sur lequel Logstash s'exécutera et établissons une connexion avec Elasticsearch, lui permettant ainsi d'y envoyer les logs.

Logstash.conf

```
input {
  beats {
    port => 5044
  }
}

output {
  elasticsearch {
    hosts => ["elasticsearch:9200"]
    index => "logs-%{+YYYY.mm.dd}"
  }
}
```

3.3. Gestion de la sécurité

Lors du transfert des données collectées depuis les serveurs surveillés vers le serveur central, celles-ci doivent être chiffrées afin de ne pas être exposées à des vulnérabilités.

Le protocole HTTPS est la version sécurisée du protocole HTTP ; il est associé au protocole TLS, ce qui permet d'établir une

connexion cryptée et garantit ainsi la protection des données transmises.

Pour activer le protocole HTTPS, nous devons créer des certificats TLS.

Nous utiliserons la technologie Vault pour émettre ces certificats.

3.3.1.Création des rôles

Dans Vault, avant d'émettre des certificats, nous créons un rôle qui définit les paramètres des certificats, tels que les domaines pour lesquels l'émission d'un certificat est autorisée, l'adresse IP et la durée maximale de validité du certificat...

```
vault write pki/roles/exporter \  
  allowed_domains="exporter" \  
  allow_ip_sans=true \  
  allow_bare_domains=true \  
  max_ttl=8760h  
  
vault write pki/roles/prometheus \  
  allowed_domains="prometheus" \  
  allow_bare_domains=true \  
  max_ttl=8760h  
  
vault write pki/roles/filebeat \  
  allowed_domains="filebeat" \  
  max_ttl=8760h
```

```
allow_bare_domains=true \  
max_ttl=8760h  
  
vault write pki/roles/logstash \  
  allowed_domains="logstash " \  
  allow_ip_sans=true \  
  allow_bare_domains=true \  
  max_ttl=8760h
```

3.3.2.Emettre des certificats

```
vault write -format=json pki/roles/exporter \  
  common_name="exporter" \  
  ip_sans="84.8.216.29" \  
  max_ttl=8760h > service_exporter.json  
  
vault write -format=json pki/roles/prometheus \  
  common_name="prometheus" \  
  max_ttl=8760h > prometheus.json  
  
vault write -format=json pki/roles/logstash \  
  common_name="logstash" \  
  ttl=8760h > logstash.json  
  
vault write -format=json pki/roles/filebeat \  
  common_name="filebeat" \  
  ttl=8760h > filebeat.json
```

3.3.3.Extraire les certificats

```
cat exporter.json | jq -r '.data.certificate' >
exporter.crt

cat exporter.json | jq -r '.data.private_key' >
exporter.key

cat rometheus.json | jq -r '.data.certificate' >
rometheus.crt

cat rometheus.json | jq -r '.data.private_key' >
rometheus.key

cat logstash.json | jq -r '.data.certificate' > logstash.crt
cat logstash.json | jq -r '.data.private_key' > logstash.key

cat filebeat.json | jq -r '.data.certificate' > filebeat.crt
cat filebeat.json | jq -r '.data.private_key' > filebeat.key
```

3.3.4.Distribution des Certificats

Pour prometheus :

```
//en prometheus.yml sous la section scrap_config
  scheme: https
  tls_config:
    ca_file: /etc/prometheus/certs/ca-chain.crt
    cert_file: /etc/prometheus/certs/prometheus.crt
    key_file: /etc/prometheus/certs/prometheus.key

/ /en docker compose sous volumes du service prometheus
- ./certs:/etc/prometheus/certs:ro
```

Pour logstash :

```
//en logstash.conf
ssl_enabled => true
  ssl_certificate =>
    "/usr/share/logstash/config/certs/logstash.crt"
  ssl_key =>
    "/usr/share/logstash/config/certs/logstash.key"
  ssl_client_authentication => "required"
  ssl_certificate_authorities =>
    ["/usr/share/logstash/config/certs/ca.crt"]
```

Pour filebeat:

```
//en filebeat.yml
ssl.enabled: true
  ssl.certificate_authorities:
    ["/etc/filebeat/certs/ca.crt"]
  ssl.certificate: "/etc/filebeat/certs/filebeat.crt"
  ssl.key: "/etc/filebeat/certs/filebeat.key"
```

Pour l'exporter:

```
//dans un fichier web-config
tls_server_config:
  cert_file: /etc/node-exporter/certs/node-exporter.crt
  key_file: /etc/node-exporter/certs/node-exporter.key

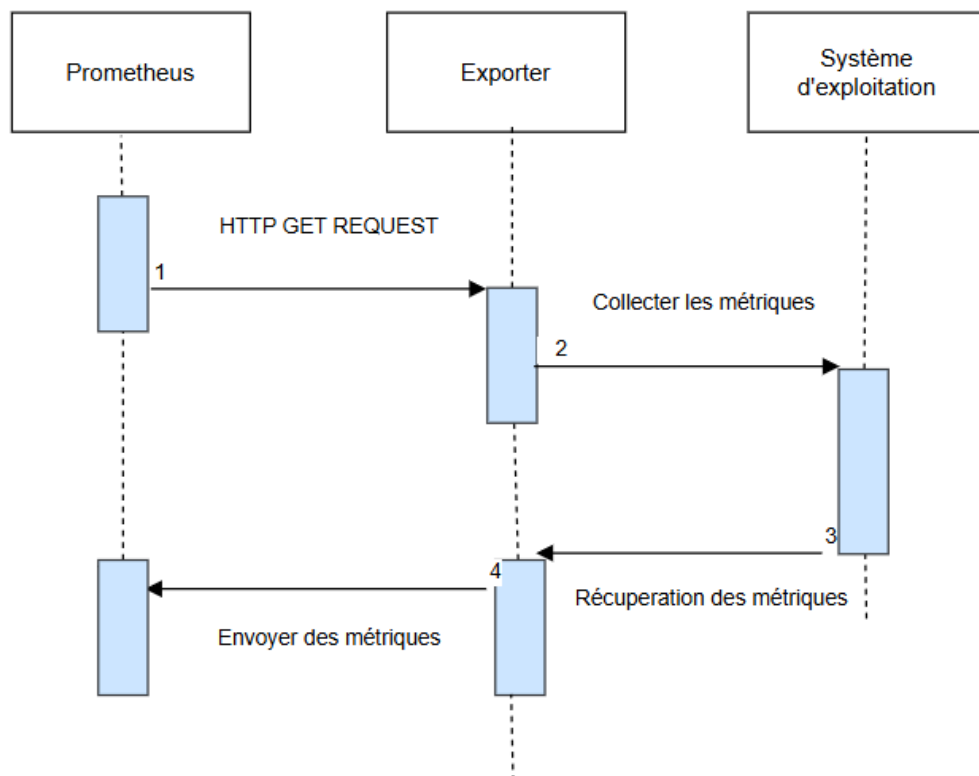
//puis l'exécuté avec le service
ExecStart=/usr/local/bin/service_exporter \
  --web.config.file=/etc/service-exporter/web-
  config.yml
```

3.4. Diagrammes techniques

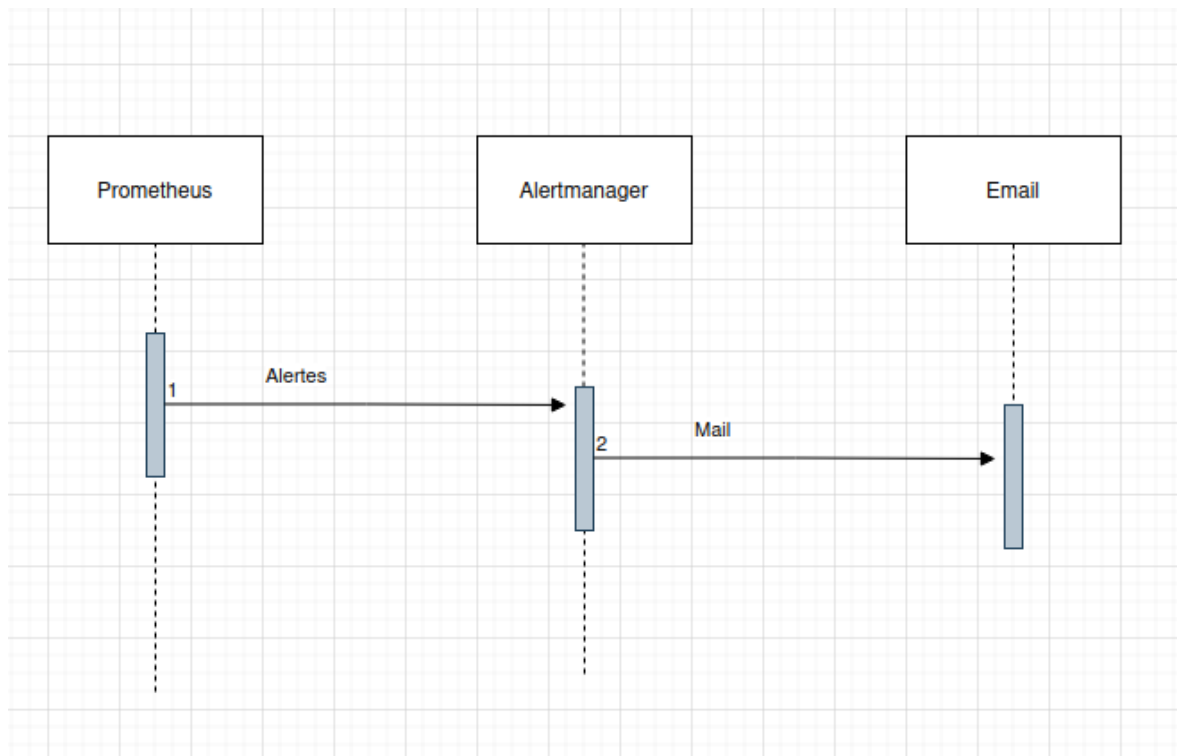
3.4.1. Diagramme de séquence

Un diagramme de séquence représente les interactions entre les éléments du système au fil du temps, selon un ordre chronologique.

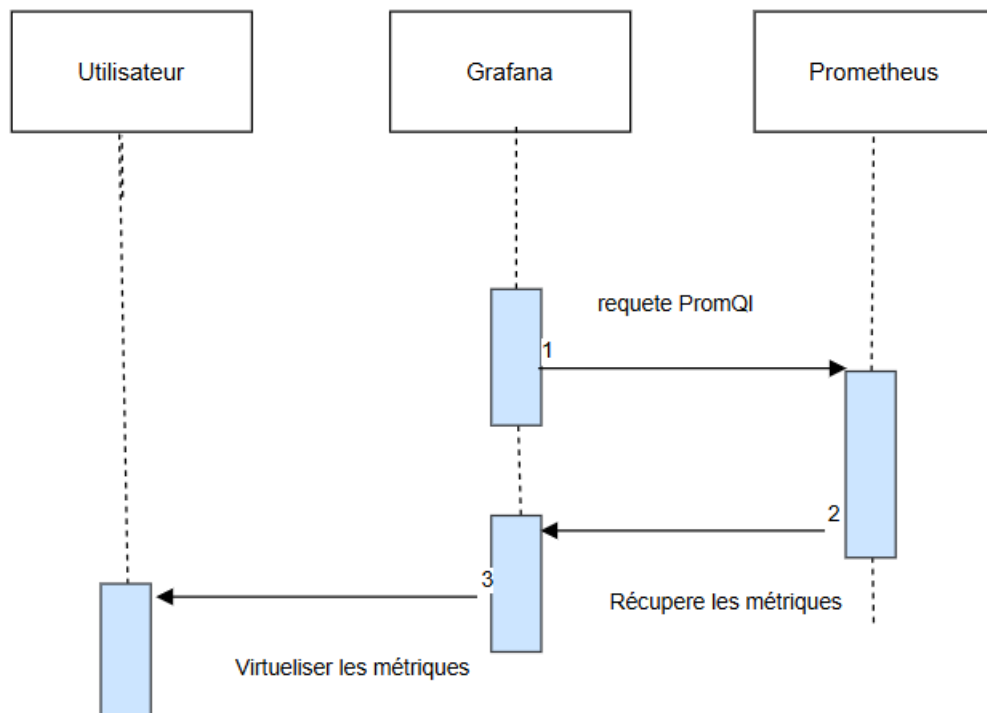
3.4.1.1. Scraping



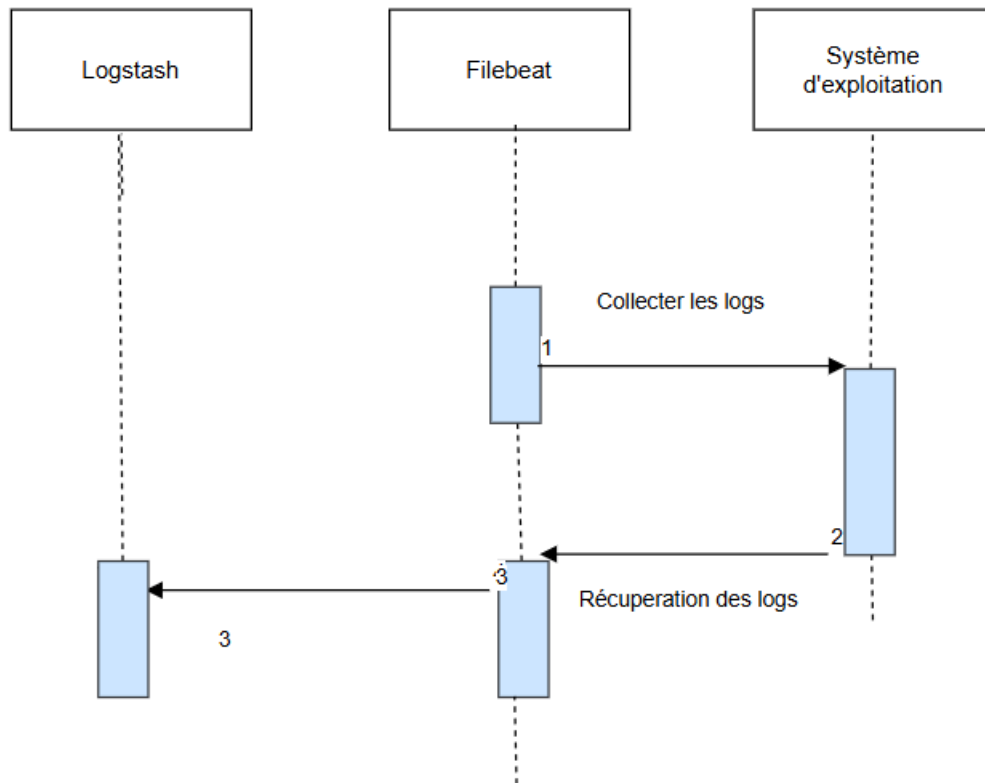
3.4.1.2. Alerting



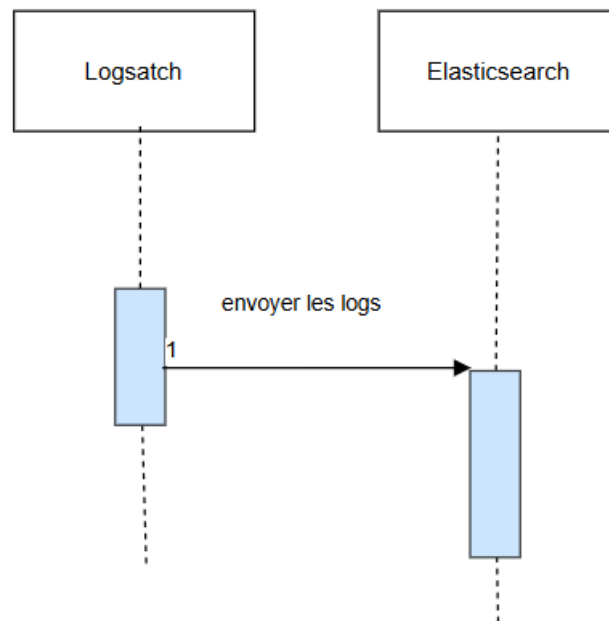
3.4.1.3. Visualisation



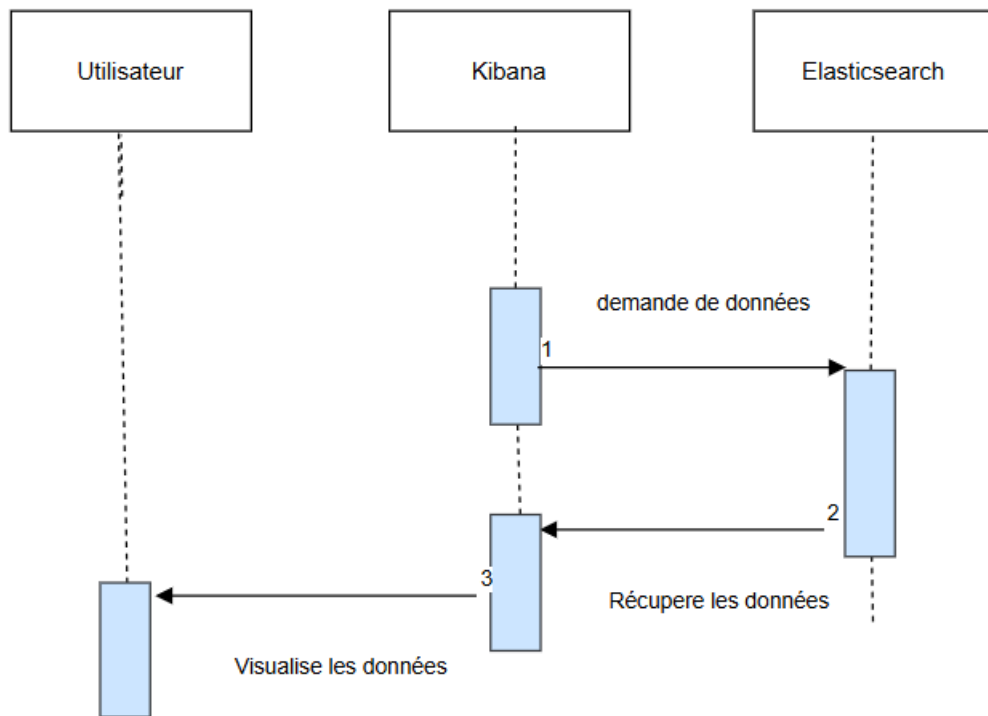
3.4.1.4. Collecte des logs



3.4.1.5. Sauvegarde des logs

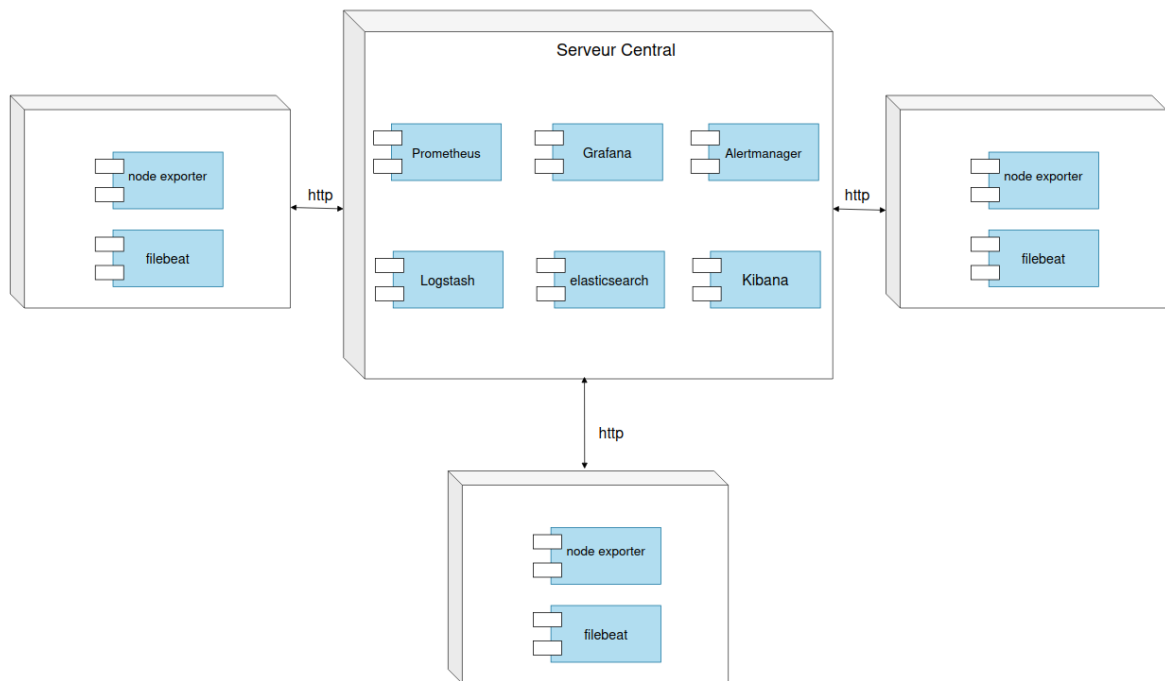


3.4.1.6. Visualisation des logs



3.4.2. Diagramme de déploiement

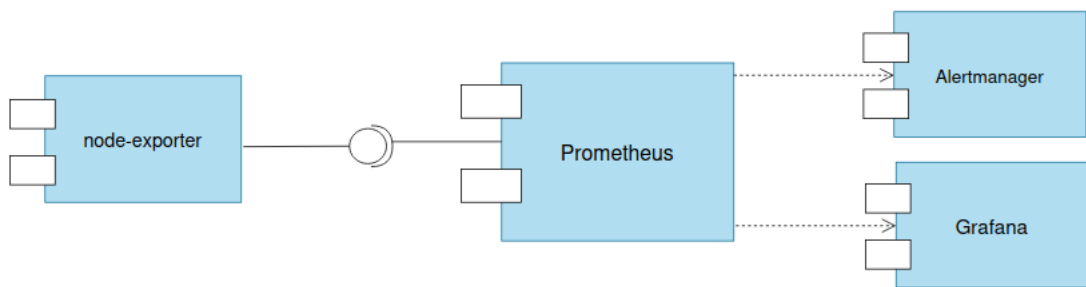
Un diagramme de déploiement représente les composants matériels du système et la manière dont ils sont répartis, ainsi que les connexions entre eux.



3.4.3. Diagramme de composants

Un diagramme des composants permet de visualiser l'organisation du système en représentant sa structure et en mettant en évidence les relations de dépendance entre les composants.

3.4.3.1. Stack Prometheus



3.4.3.2. Stack ELK



Conclusion

Ce chapitre a marqué la réalisation de ce projet : sa structure a été mise en place, ses outils ont été installés et configurés, ce qui a permis de le mettre en service et de le rendre opérationnel.

Chapitre 4 : Expérimentation et résultats

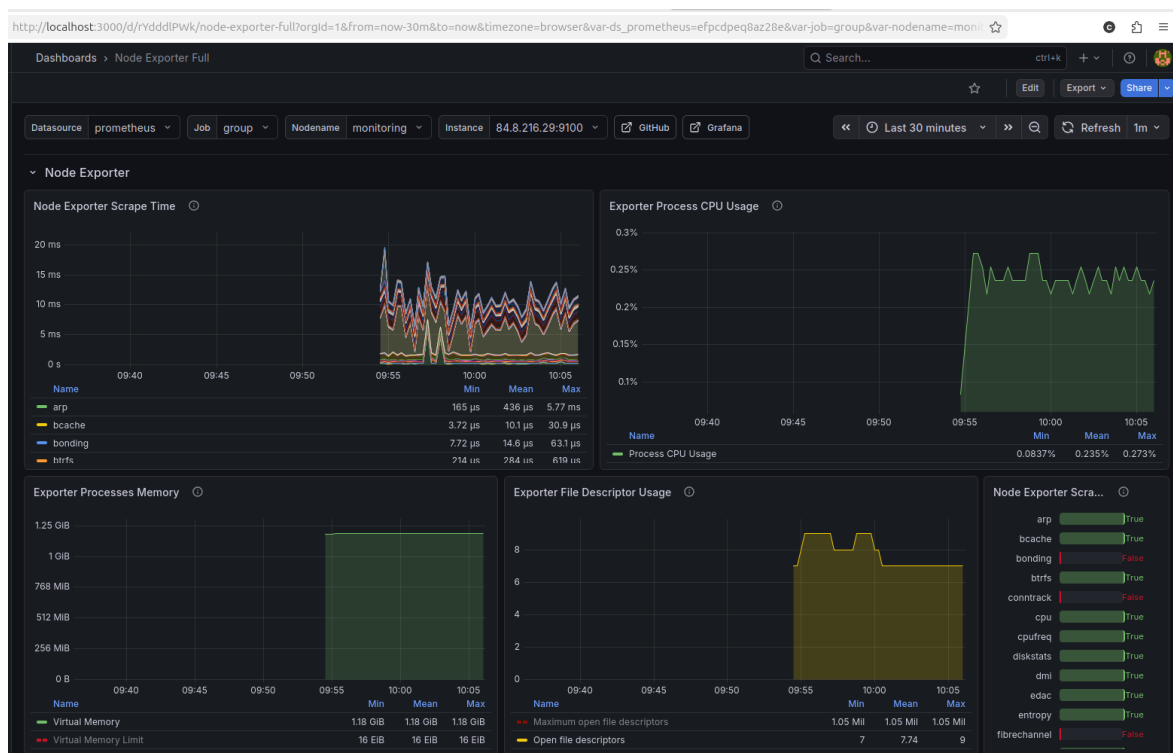
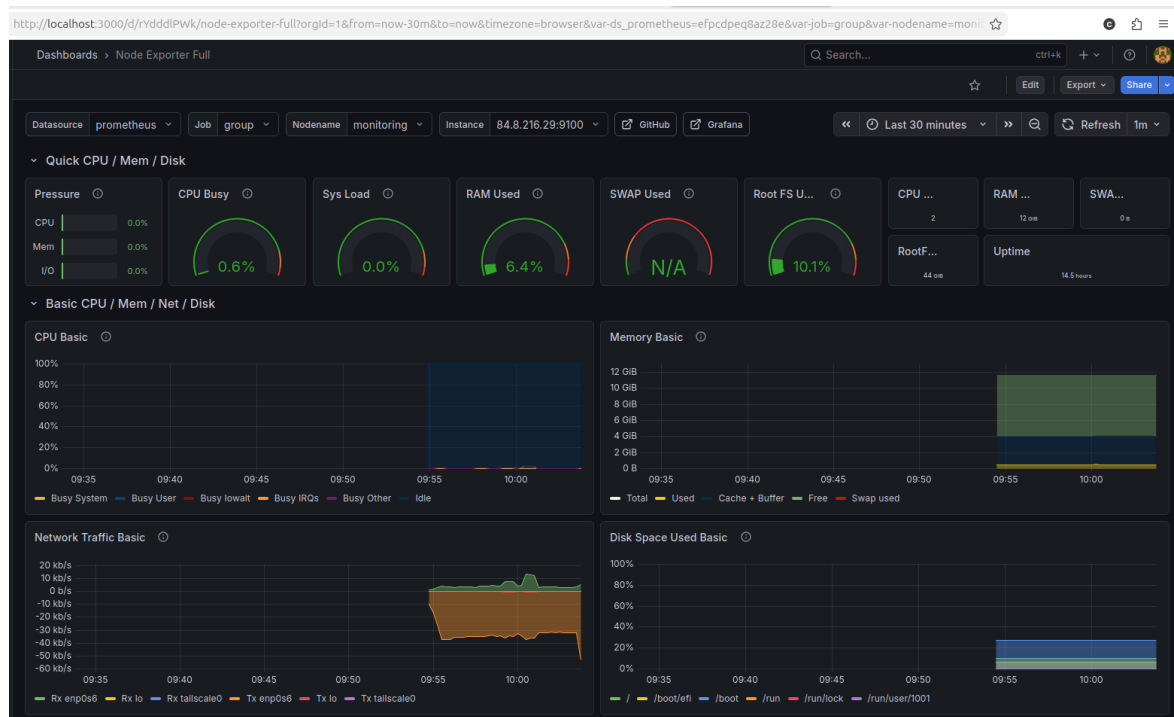
Intorduction

Ce chapitre portera sur les résultats obtenus grâce à la mise en œuvre de ce projet et à son test, afin de vérifier sa stabilité et sa validité.

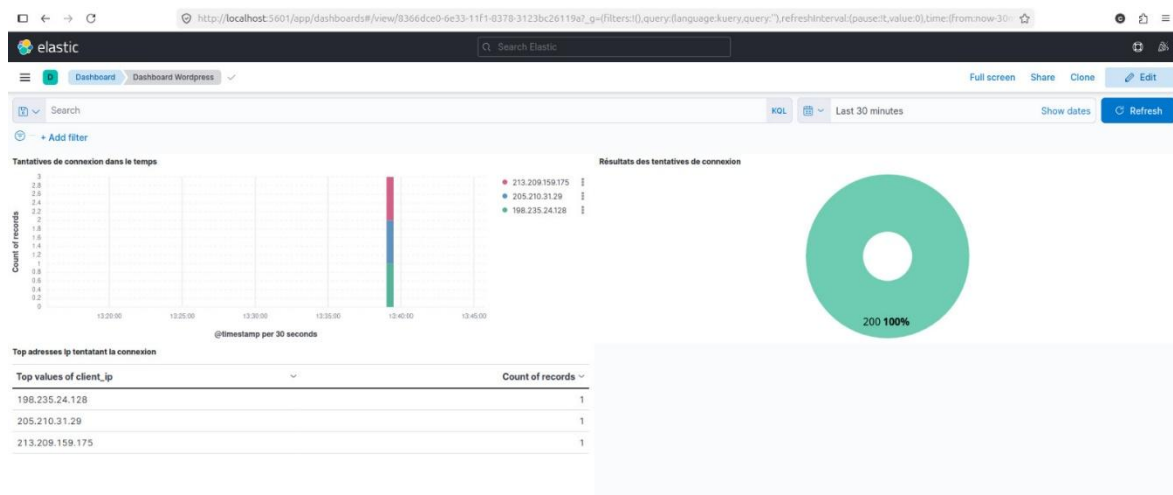
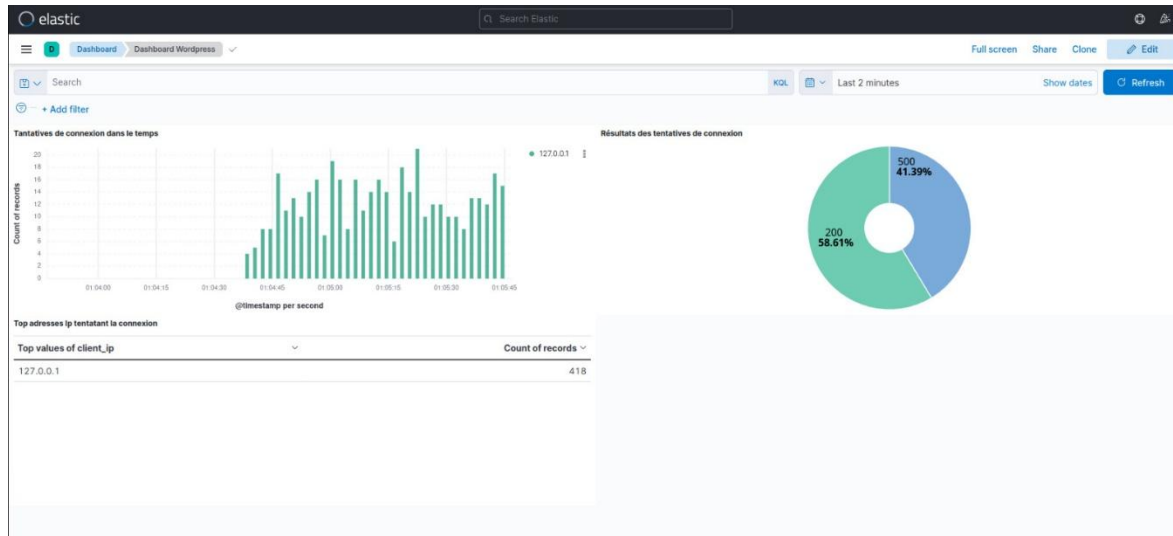
Pour expérimenter notre système, nous utilisons une machine virtuelle (VM) d'Oracle en tant que serveur surveillé, équipée du système d'exploitation Ubuntu, de 12 Go de RAM, d'un CPU double cœur et d'un disque dur de 240 Go. Nous y exécutons l'application web « WordPress » en tant que service surveillé avec une base de données MySQL. Nous utilisons également notre ordinateur local comme serveur central.

Sur ce serveur surveillé, nous installons Filebeat et Node-Exporter, puis nous configurons les deux cibles sur notre serveur central.

4.1. Tableau de bord Grafana



4.2. Tableau de bord Kibana

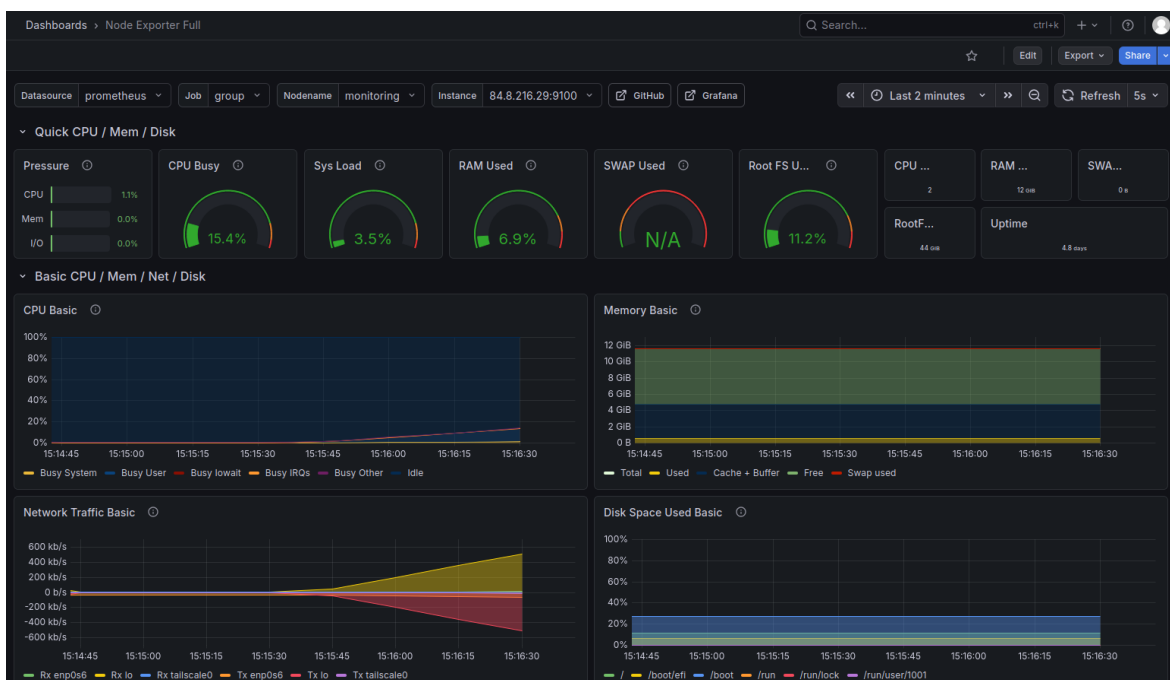
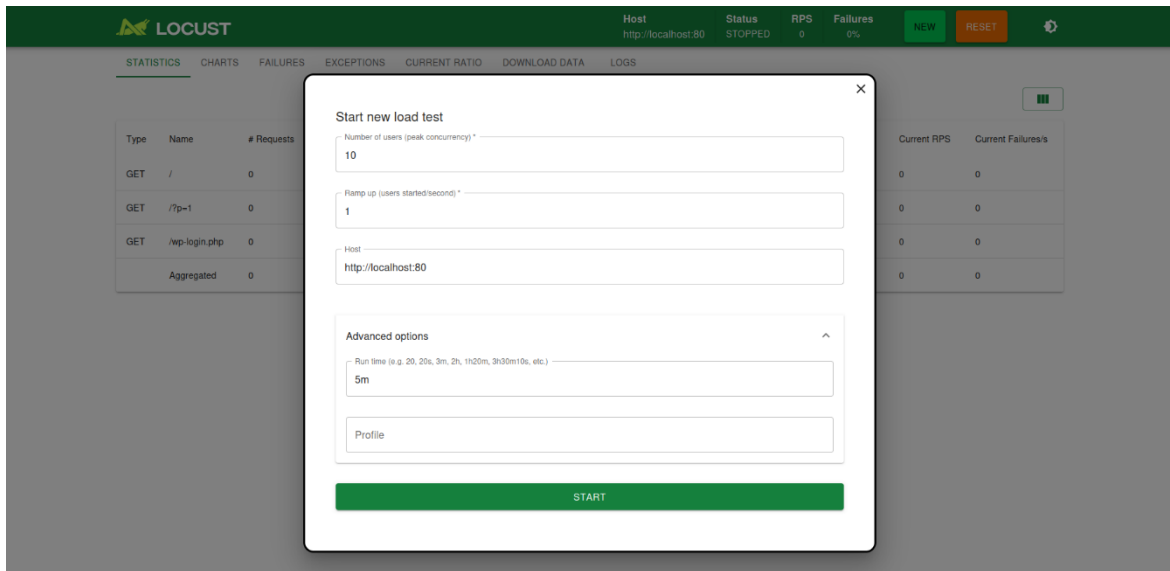


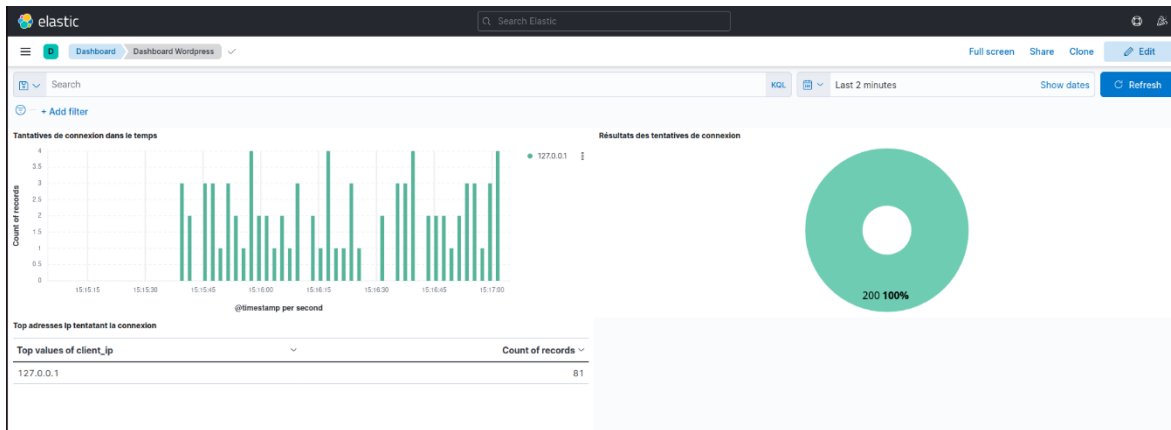
4.3. Tests de charge et analyse de performance

Afin d'évaluer les performances du système de surveillance en conditions de charge, l'outil Locust a été utilisé pour tester la

charge sur le service WordPress qui a été choisi comme service à surveiller.

❖ Teste 1





LOCUST

Host

http://localhost:80

Status

STOPPED

RPS

4.73

Failures

0%

NEW

RESET



STATISTICS

CHARTS

FAILURES

EXCEPTIONS

CURRENT RATIO

DOWNLOAD DATA

LOGS



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/	358	0	61	67	77	62.12	58	105	72705	2.6	0
GET	/?p=1	201	0	70	77	87	70.78	67	121	72394	1	0
GET	/wp-login.php	149	0	22	26	42	22.61	20	43	8117	1.3	0
	Aggregated	708	0	62	73	80	56.27	20	121	59024.04	4.9	0

Avec 10 utilisateurs, le système a enregistré un temps de réponse moyen de 56.27 ms, avec une RPS 4.9, sans qu'aucune erreur ne soit signalée. Le tableau de bord Grafana a montré une utilisation normale des ressources, ce qui confirme que le système gère efficacement les charges légères.

❖ Teste 2

×

Start new load test

Number of users (peak concurrency) *

50

Ramp up (users started/second) *

5

Host

http://localhost:80

Advanced options

^

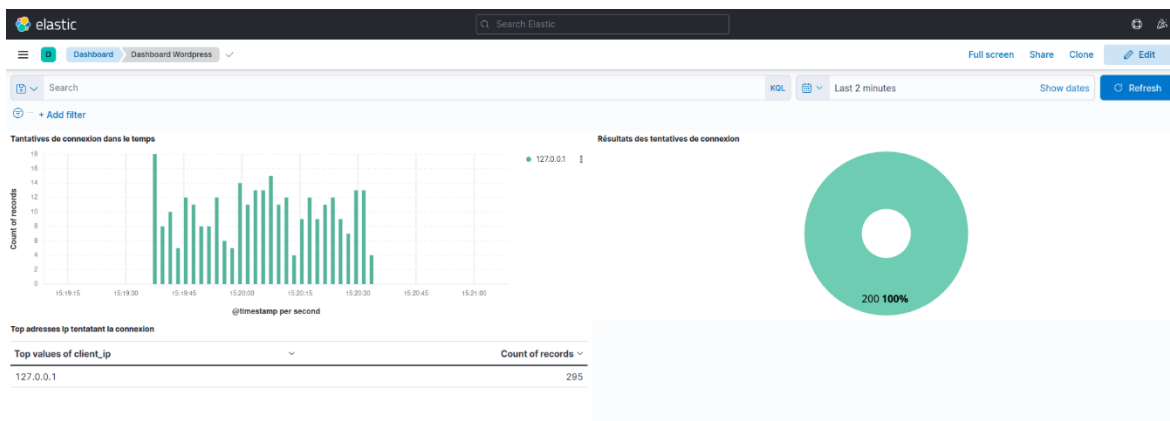
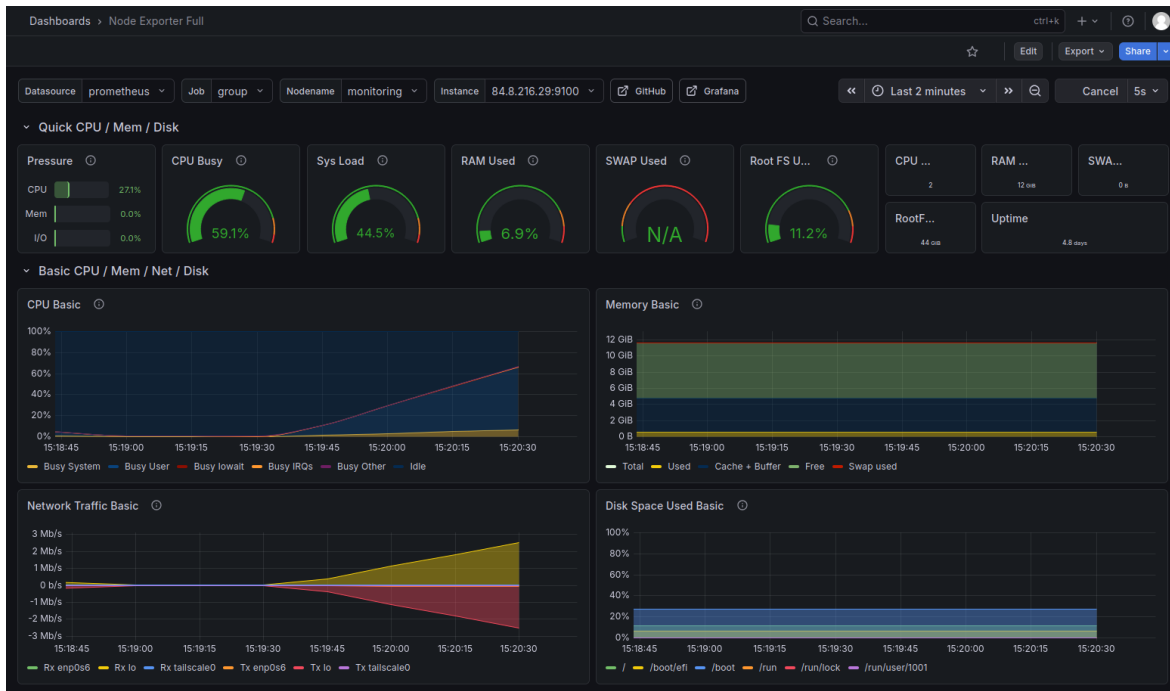
Run time (e.g. 20, 20s, 3m, 2h, 1h20m, 3h30m10s, etc.)

2m

Profile

START

Chapitre 4: Expérimentation et résultats



 LOCUST

Host

http://localhost:80

Status

STOPPED

RPS

22.34

Failures

0%

NEW

RESET



STATISTICS

CHARTS

FAILURES

EXCEPTIONS

CURRENT RATIO

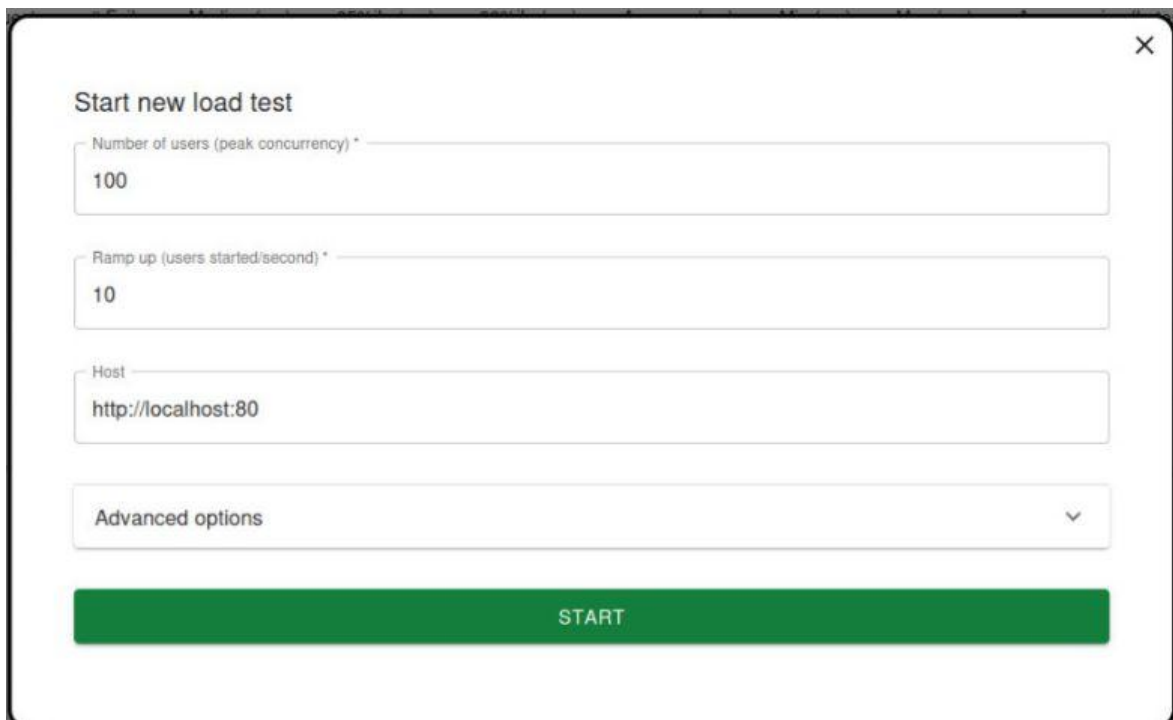
DOWNLOAD DATA

LOGS

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/	721	0	89	250	330	115.46	59	403	72705	11.8	0
GET	/?p=1	397	0	110	270	380	130.32	67	414	72394	6	0
GET	/wp-login.php	295	0	29	150	230	47.79	20	245	8117	5.5	0
Aggregated		1413	0	83	240	330	105.51	20	414	59133.22	23.3	0

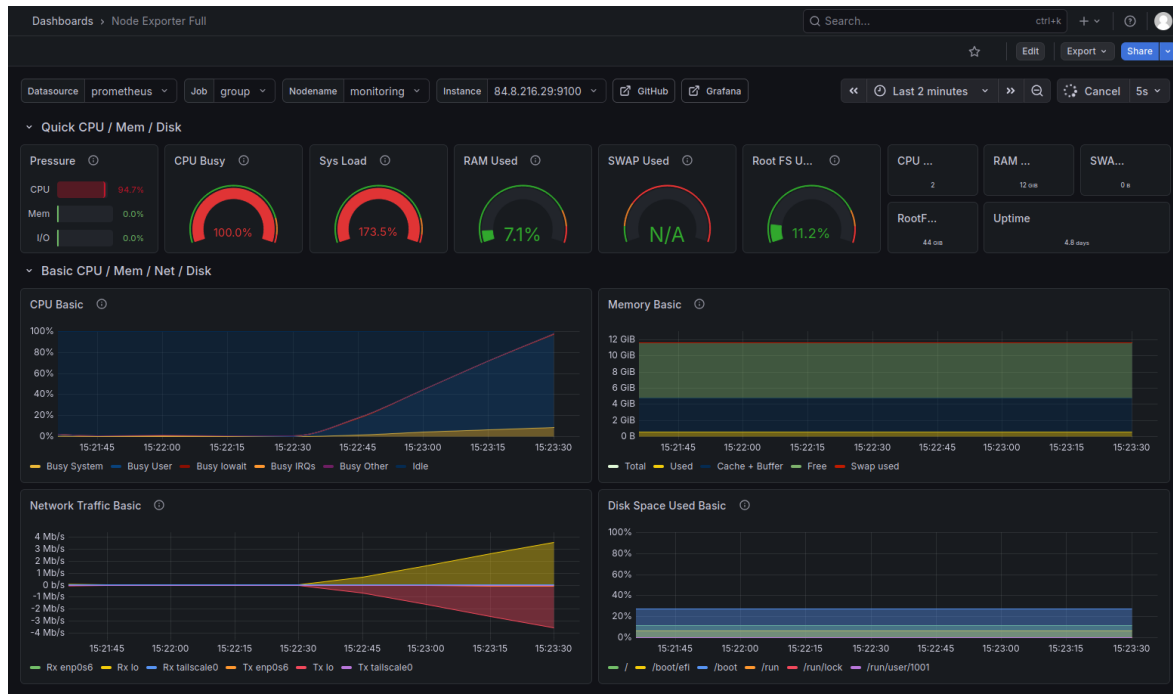
Avec 50 utilisateurs, le temps de réponse est passé à 105.51 ms, avec une RPS égale à 23.3 . Le tableau de bord Grafana a enregistré une augmentation progressive de l'utilisation du processeur, tandis que Kibana a continué à enregistrer les journaux en temps réel sans interruption.

❖ Teste 3



The image shows a 'Start new load test' dialog box with the following fields and values:

- Number of users (peak concurrency) ***: 100
- Ramp up (users started/second) ***: 10
- Host**: http://localhost:80
- Advanced options**: (dropdown menu with a downward arrow)
- START**: (green button)



1 alert for

View In Alertmanager

[1] Firing

Labels

alertname = HighLoadAverage

instance = [84.8.216.29:9100](#)

job = group

severity = warning

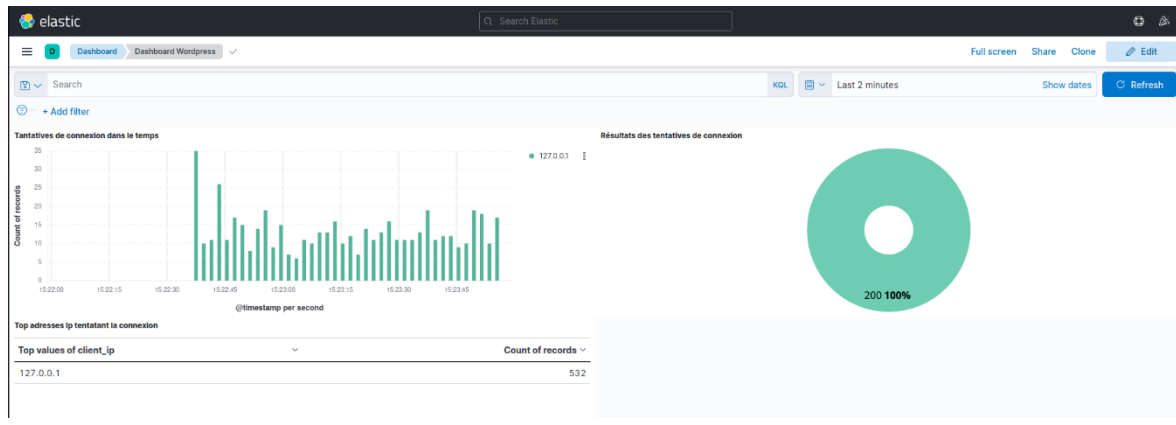
Annotations

description = Le charge moyenne du système est élevée

summary = High Load Average

[Source](#)

Sent by Alertmanager



LOCUST

Host

http://localhost:80

Status

STOPPED

RPS

31.61

Failures

0%

NEW

RESET



STATISTICS

CHARTS

FAILURES

EXCEPTIONS

CURRENT RATIO

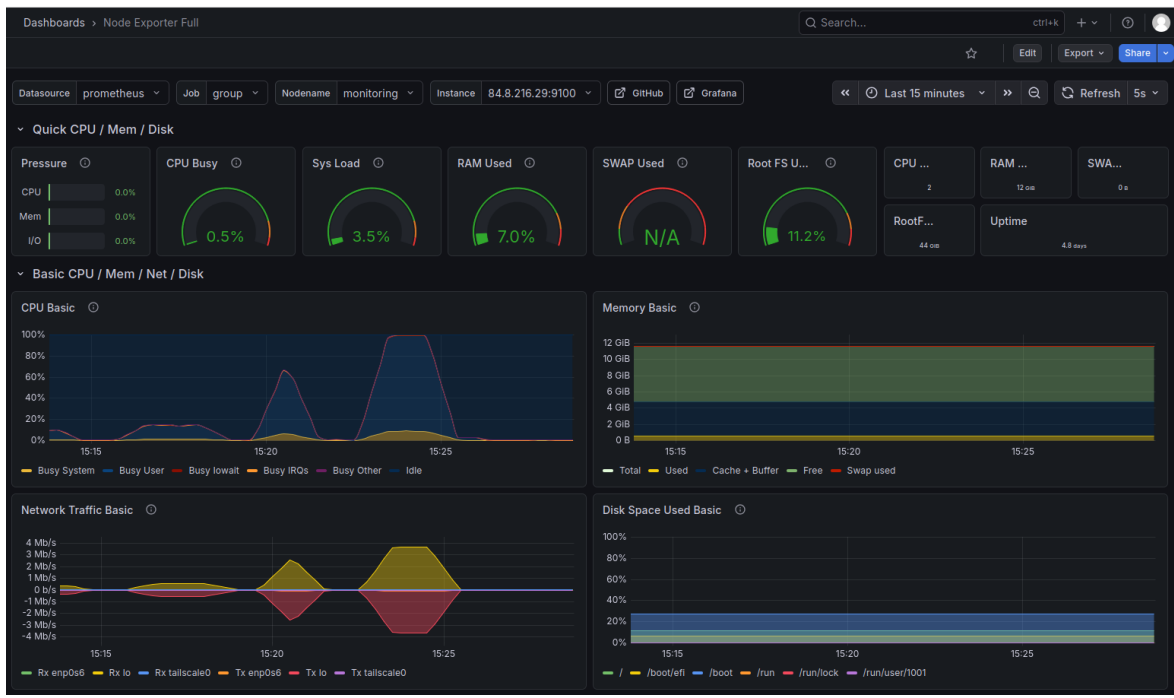
DOWNLOAD DATA

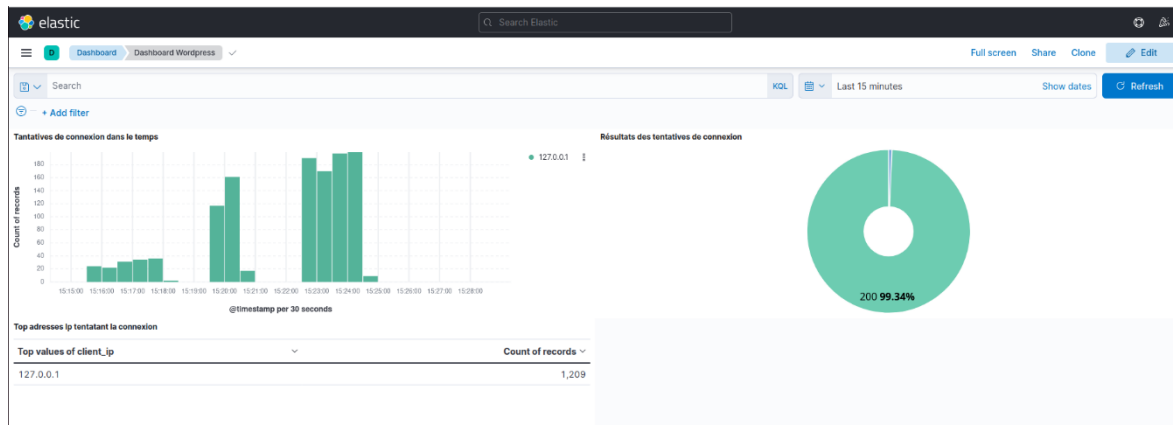
LOGS



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/	1887	0	1100	1400	1500	1080.64	59	1557	72705	15.9	0
GET	/?p=1	1161	0	1100	1400	1500	1108.75	69	1558	72394	9.4	0
GET	/wp-login.php	757	0	1000	1300	1300	974.41	21	1398	8117	7.4	0
	Aggregated	3805	0	1100	1400	1500	1068.08	21	1558	59760.4	32.7	0

Avec 100 utilisateurs, l'utilisation du processeur a dépassé 100 % selon Grafana, ce qui s'est clairement traduit par une augmentation du temps de réponse à 1068.08 ms contre 56.27 ms lors de la première phase, indiquant un début de saturation des ressources et un ralentissement notable dans le traitement des requêtes.





2 alerts for

View In Alertmanager

[2] Resolved

Labels

alertname = HighLoadAverage
instance = [84.8.216.29:9100](#)
job = group
severity = warning

Annotations

description = Le charge moyenne du système est élevée
summary = High Load Average
[Source](#)

Labels

alertname = HighCPUUsage
instance = [84.8.216.29:9100](#)
severity = critical

Annotations

description = L'utilisation du CPU a augmentée de 80%
summary = High CPU Usage
[Source](#)

Les résultats du test de charge ont démontré que notre système de surveillance était capable de surveiller l'état des services en temps réel tout au long des différentes phases du test. Alors que

la charge passait de 10 à 100 utilisateurs simultanés, les tableaux de bord Grafana ont continué d'afficher en continu les indicateurs de ressources, et Kibana a enregistré le flux de logs sans interruption, ce qui a permis de suivre la dégradation des performances en temps réel.

Plus important encore, le système de surveillance a détecté à l'avance une augmentation de l'utilisation du processeur à 80 % à 100 utilisateurs, un indicateur d'alerte précoce permettant aux administrateurs d'intervenir avant que le système n'atteigne le point de rupture.

Conclusion

Grâce à ces valeurs, nous avons pu nous assurer de la mise en œuvre du projet et de la réalisation de son objectif, qui permet effectivement d'assurer la surveillance et nous tient informés de l'état des serveurs et des service

Chapitre 5 : Discussion

Introduction

Dans ce chapitre, nous aborderons de manière générale l'avenir de ce projet, sa capacité à se pérenniser et à se développer, les contraintes auxquelles il est confronté, les difficultés que nous avons rencontrées lors de sa mise en œuvre, ainsi que les moyens de les traiter et de les surmonter.

4.1. Limites de l'approche proposée

Même si ce système que nous avons mis en place résout de nombreux problèmes et apporte des solutions exemplaires à bon nombre des défis auxquels sont confrontés les services publics, il a toutefois ses limites.

L'une des principales limites réside dans le fait que Prometheus n'est pas conçu pour conserver les données sur de long terme, c'est-à-dire pendant plusieurs années. Par défaut, il les conserve pendant quinze jours, puis les supprime de sa base de données et du disque dur. Et bien que la durée de conservation puisse être réglée sur plusieurs mois, un an ou plus, dès que sa base de données est pleine, le système commence à ralentir et à présenter des retards.

Les times séries dans Prometheus sont composées de métriques et de valeurs de labels. Prometheus les stocke en RAM sous forme d'index. Ainsi, l'ajout d'un grand nombre de labels entraîne une multiplication des times séries, ce qui traduit par une

augmentation de la consommation de RAM et de l'utilisation du processeur, ainsi que par un allongement de la durée des requêtes PromQl.

4.2. Problèmes rencontrés et solutions apportées

Au cours de la mise en œuvre de ce projet, nous avons rencontré plusieurs problèmes et défis, notamment la difficulté de migration d'un environnement à un autre lors de l'exécution directe sur le système, ce qui nécessite de réinstaller chaque technologie depuis le début, ainsi que la compatibilité des versions un système et leurs incompatible entre elles, ce qui oblige à interrompre le service et à remplacer la version à chaque fois. La solution à ce problème a consisté à utiliser la technologie des conteneurs, chaque conteneur contenant l'application de manière isolée avec tout ce dont elle a besoin pour fonctionner, sans qu'il soit nécessaire de vérifier sa compatibilité avec le système d'exploitation. De plus, lors du transfert vers un autre environnement, le transfert du conteneur est très flexible.

Nous avons également rencontré une autre difficulté due à notre manque de connaissance préalable de PromQl, mais celle-ci a été surmontée en nous appuyant sur la documentation de Prometheus.

De plus, Nous avons rencontré un problème lors de l'utilisation de la machine virtuelle Oracle en raison d'une différence de réseau entre les deux appareils, mais nous avons réussi à le résoudre grâce à la technologie Tailscale.

4.3. Scalabilité et haute disponibilité

Dans les grands systèmes formels, la scalabilité est une possibilité qui doit toujours être envisagée et prise en compte. Comme nous attendrons de notre système qu'il soit stable et pérenne, nous devons tenir compte de l'augmentation potentielle du nombre de serveurs surveillés et les services qu'ils héberges au fil du temps.

Prometheus prend en charge la scalabilité grâce à un mécanisme de fédération qui repose sur l'agrégation hiérarchique dans lequel un Prometheus parent récupère des métriques à partir de plusieurs Prometheus enfants.

Au niveau du ELk la scalabilité est traitée par l'augmentation du nombre des nodes dans cluster en divisant les données en "shards" et les partagés entre les nodes cela permet de traiter un grand nombre des données.

Une haute disponibilité est également indispensable dans les grandes structures sensibles, où aucune panne ni interruption

n'est tolérée ; il est donc nécessaire de disposer d'un mécanisme garantissant la continuité de la collecte des métriques et de l'envoi des alertes. C'est pourquoi il convient de déployer plusieurs instances de Prometheus avec mêmes configuration si l'une d'entre elles tombe en panne ou cesse de fonctionner, l'autre continue de fonctionner. Et en ELK Ajouter plusieurs nodes à un cluster Elasticsearch et en désigner un comme réplique des données.

Conclusion

Ce chapitre offrait un aperçu général du projet une fois celui-ci achevé, en présentant ses possibilités et ses limites.

Conclusion et perspectives

En conclusion, ce projet visait à améliorer la qualité des services publics en surveiller les serveurs qui les hébergés à l'aide d'un système reposant sur les meilleurs outils de surveillance, tels que Prometheus, Grafana, ELK après les avoir étudiés, approfondi leur compréhension et les avoir comparés à d'autres alternatives ; puis en les configurant selon des critères précis adaptés à leur environnement d'exploitation final.

Ce projet nous a permis de découvrir l'importance de définir une architecture pour le système et de la respecter, d'étudier les besoins et les exigences avant sa mise en place, puis d'apporter des améliorations et des modifications par la suite ; ainsi que le choix des outils, après les avoir étudiés et sélectionnés parmi les différentes options disponibles, ce qui nous permet de découvrir et d'acquérir des connaissances sur de nombreuses technologies dans ce domaine.

Nous avons mise en place Ce système en plusieurs étapes, en commençant par mener des recherche et des études sur un certain nombre de technologies utilisées dans le domaine de la surveillance, puis en choisissant Prometheus et Grafana parmi celles-ci pour la surveillance du serveur et ELK pour les services, ensuite, nous avons élaboré une architecture générale du système adaptée à la structure du ministère, qui consiste l'environnement opérationnel de ce système, puis nous avons étudié les différentes méthodes de configuration de ces outils et sélectionné celles qui nous convenaient, en mettant l'accent sur les aspects liés aux performances et à l'évolutivité, enfin, nous

avons testé le système pour nous assurer de sa flexibilité et de sa stabilité.

Nous espérons que ce projet constituera un atout pour nos services publics en aidant les administrateurs à connaître parfaitement l'état des serveurs et services, ce qui leur permettra de détecter les erreurs et d'intervenir en temps opportun. Il améliorera également l'expérience utilisateur de ces services publics en réduisant le nombre d'erreurs rencontrées ou en les résolvant avant même qu'elles ne survie.

Références

Bibliographie

- [1] <https://www.ibm.com/fr-fr/think/topics/virtualization?regionCode=fr&languageCode=fr&cm-history=fr-fr>
- [2] <https://fr.wikipedia.org/wiki/Chroot>
- [3] <https://fr.wikipedia.org/wiki/Virtualisation>
- [4] [https://en.wikipedia.org/wiki/Containerization_\(computing\)](https://en.wikipedia.org/wiki/Containerization_(computing))
- [5] [https://en.wikipedia.org/wiki/Docker_\(software\)](https://en.wikipedia.org/wiki/Docker_(software))
- [6] <https://www.aquasec.com/blog/kubernetes-history-how-it-conquered-cloud-native-orchestration/>

Webographie

Numéro	Lien
01	https://en.wikipedia.org/wiki/Timeline_of_virtualization_technologies
02	https://www.ibm.com/fr-fr/think/topics/virtualisation?regionCode=fr&languageCode=fr&cm-history=fr-fr
03	https://www.redhat.com/fr/topics/virtualization/what-is-virtualisation
04	https://www.coursera.org/fr-FR/articles/containerization
05	https://www.redhat.com/fr/topics/cloud-native-apps/what-is-containerization
06	https://www.coursera.org/articles/what-is-a-virtual-machine
07	https://aws.amazon.com/fr/compare/the-difference-between-containers-and-virtual-machines/
08	https://6sense.com/tech/containerization
09	https://docs.docker.com/get-started/docker-overview
10	https://www.ibm.com/think/topics/docker
11	https://www.oracle.com/africa/cloud-native/container-registry/what-is-

	docker/
12	https://www.redhat.com/en/topics/containers/what-is-podman
13	https://docs.podman.io/en/latest/
14	https://en.wikipedia.org/wiki/Podman
15	https://linuxcontainers.org/lxc/introduction/
16	https://www.redhat.com/fr/topics/containers/whats-a-linux-container
17	https://fr.wikipedia.org/wiki/LXC
18	https://www.ibm.com/fr-fr/think/topics/linux-containers?regionCode=fr&languageCode=fr&cm-history=fr-fr
19	https://prometheus.io/docs/introduction/overview/
20	https://liora.io/prometheus-tout-savoir
21	https://fr.wikipedia.org/wiki/Prometheus_(logiciel)
22	https://www.oracle.com/fr/security/nagios-logiciel-surveillance/
23	https://fr.wikipedia.org/wiki/nagios
24	https://liora.io/nagios-tout-savoir
25	https://www.zabbix.com/documentation/1.8/fr/manual/about/overview_of_zabbix
26	https://lingora.com/technology/zabbix
27	https://prometheus.io/docs/guides/node-exporter/
28	https://prometheus.io/docs/visualization/grafana/
29	https://prometheus.io/docs/alerting/latest/alertmanager/
30	https://prometheus.io/docs/prometheus/latest/federation/
31	https://prometheus.io/docs/prometheus/latest/configuration/configuration/
32	https://www.oracle.com/fr/database/elk-donnees-logs/

ANNEXE

Annexe A: Attestation de stage

الجمهورية الإسلامية الموريتانية
شرف - إخاء - عدل

RÉPUBLIQUE ISLAMIQUE DE MAURITANIE
Honneur - Fraternité - Justice

وزارة التحول الرقمي وعصرية الإدارة
Ministère de la Transformation Numérique et de la
Modernisation de l'Administration
مديرية التطوير والتشغيل البيئي
Direction du Développement et de l'interopérabilité



نواكشوط، le 01 JUN 2026 تراكشوط لي
Numéro 000260 الرقم

ATTESTATION DE STAGE

Je, soussigné Monsieur Mohamed Vall CHEIKH, Directeur Adjoint du Développement et de l'Interopérabilité au Ministère de la Transformation Numérique et de la Modernisation de l'Administration

Atteste que :

L'étudiante Chivae Ahmed Bezeid, inscrite en 3^{ème} année de Licence en Méthodes Informatiques Appliquées à la Gestion des Entreprises (MIAGE), a effectué un stage au sein de notre direction, sous le thème : « Mise en place d'un système de monitoring et d'alerting des services et des serveurs publics », durant la période du 01 mars au 01 juin 2026.

En foi de quoi, la présente attestation lui est délivrée pour servir et valoir ce que de droit.

Mohamed Vall CHEIKH


Direction du Développement et de l'interopérabilité
Ministère de la Transformation Numérique et de la Modernisation de l'Administration

Annexe B: mes profils professionnels

- **Compte LinkedIn :** Chivae Ahmed Bezeid
- **Compte GitHub :** <https://github.com/chivaebzd>